

8/22/23

Lecture 1 - Intro

Further reading MLPP ch1 & ESL ch1 & 2

AI is a computer system that mimics human intelligence, like taking a stimuli from its surroundings and make decisions.

ML takes input data and makes predictions using algorithms, mathematics and statistics. ML is underpinning of AI.

ML learn from input data ~~and~~ to generalize to new data. It is not like modeling your current data, but you use ML produce model to generalize to new data.

ML Setup:

Data matrix $X \in \mathbb{R}^{n \times p}$



Tabular data:

n : observations, samples instances etc.

p : features, variables,

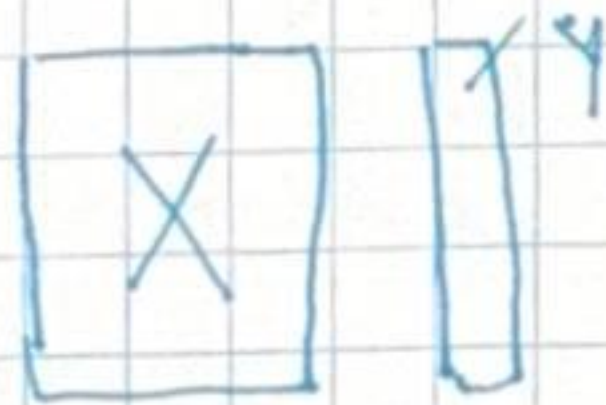
Other data that is hard to organize as tabular form.

Graphs, networks, image, text data, sequential data.

Types of ML:

1) Supervised learning:

We have some types of labels, outcomes, responses affiliated with our data $\xrightarrow{\text{which are}}$ y (outcome)



$$(x_i, y_i), i = 1, 2, \dots, n$$

* y supervises the training process

x : scalar X : vector X : matrix

a) Regression

$$y_i \in \mathbb{R}$$

* Model:

$$y_i = f(x_i) + \epsilon$$

independent noise

$$f: \mathbb{R}^p \rightarrow \mathbb{R}$$

function to learn: this maps $X \xrightarrow{\text{to}} y$

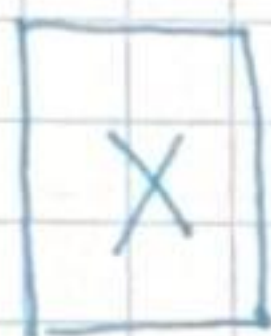
* New data: (x_i^*, y_i^*)

* Goal: Learn f from $(x_i, y_i) \rightarrow \hat{f}(\cdot)$
predict $\hat{y} = \hat{f}(x_i^*)$

b) Classification:

$$\begin{aligned} y_i &\in \{1, \dots, K\} \text{ multi-class} \\ y_i &\in \{-1, 1\} \text{ binary} \\ &\rightarrow \hat{y}_i = \text{sign}(\hat{f}(x_i^*)) \end{aligned}$$

2) Unsupervised Learning



* No labels or outcomes, no y

examples: clustering, dim-reduction, patterns, anomalies, prototypes, graphs/networks

* Goal: Find patterns or some underlying structures that generalize to new data.

3) Other types:

- Reinforcement Learning
- Semi-Supervised
- Online Learning
- Recommendation systems

} not covering these models

BASICS OF MACHINE LEARNING:

1) Motivation: Goal is modeling to generalize to future data, how? and how do we know it modeled future data?

(i) set aside some data: Training / Test split



Training set: Model fitting / training
Goal: Learn / estimate the $\hat{f}()$ function.

Test set:

* Remains hidden while training; Model evaluation only.

Goal: Prediction of $\hat{y}$
Model of evaluation is get the prediction error.!

We are trying to minimize the prediction error, on a new unseen data set.

Prediction error: $\text{Error}(y^{\text{test}}, \hat{f}(x^{\text{test}}))$

learned from $(x^{\text{tr}}, y^{\text{tr}})$

Error : (1) MSE , mean square error. $\left. \begin{array}{l} \text{used in} \\ \text{Regression} \end{array} \right\}$

$$\frac{1}{n_{\text{test}}} \| y^{\text{test}} - \hat{y} \|^2_2$$

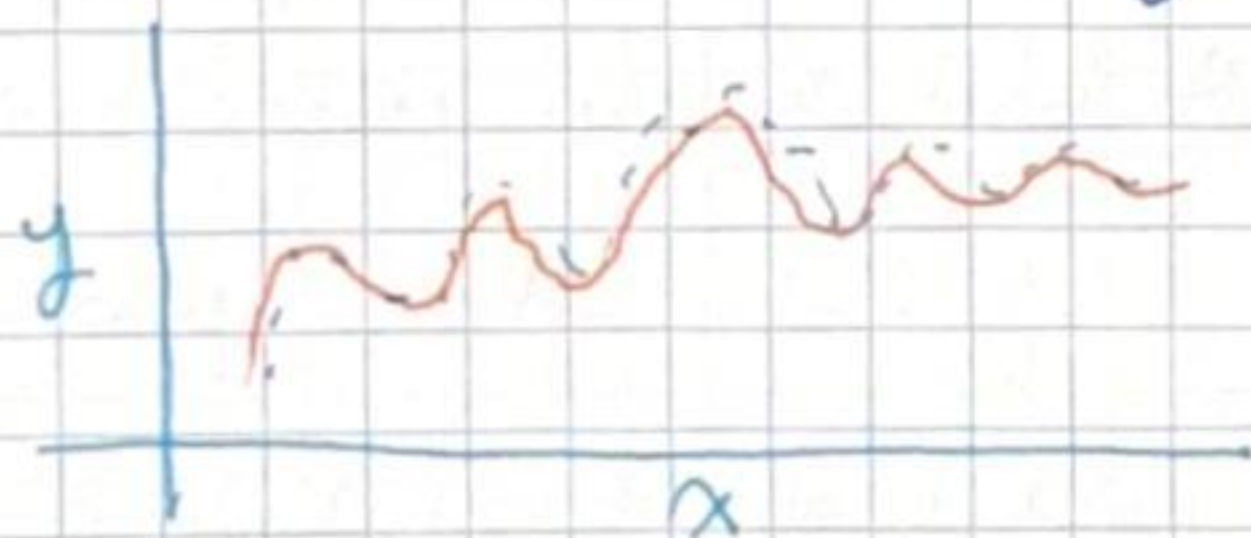
(2) Miss classification error $\left. \begin{array}{l} \text{used in} \\ \text{Classification} \end{array} \right\}$

* We split to training & test set

Training error : $\min_{f \in \mathcal{F}} \text{MSE}(y^{\text{tr}}, f(x^{\text{tr}}))$

If $\mathcal{F}$ is an expressive class, (which means can produce very wiggly functions)

that makes the training Error $\rightarrow 0$



→ interpolator.
it has minimum training error.

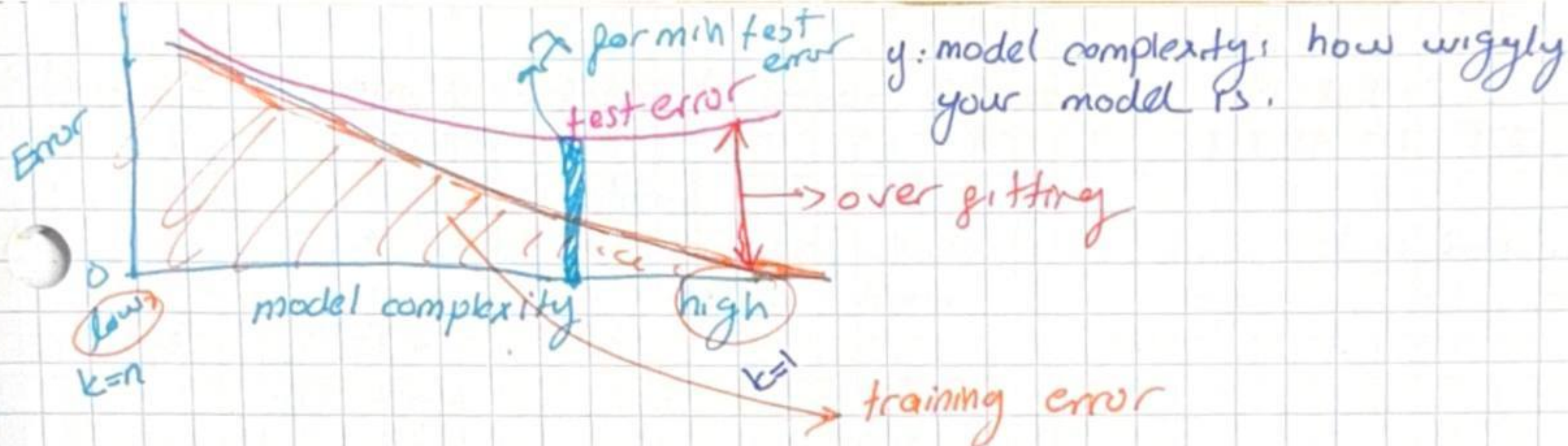
Overfitting !

⊗ When we minimize MSE using training data, that produces a model that has ~ 0 training error. This model is very unlikely to predict new data. $\Rightarrow$ That is a terrible function

⊗ Instead we should minimize the test error, we get functions that does much better.

⊗ Split your data into training and test, and minimize the fit your data by using many different hyper parameters.

⊗ We have ML families of models, each of which is indexed by hyper parameters (λ) which controls the expressiveness of $\hat{f}()$.
→ learned function
model complexity.



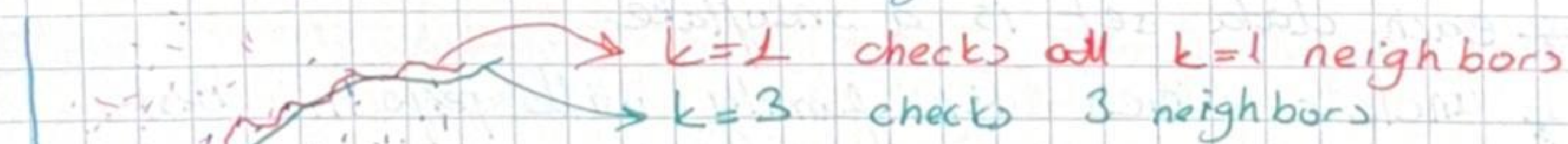
When model complexity is high, which means we have a wiggly function, training ^{error} function will be very low. However, test error will be high, because the model is not a good predictor.

Overfitting is when we over learn the training data, which means we learned the noise (ϵ) also. It doesn't generalize to new data.

Our goal is to find the λ (hyperparameters) that minimizes the test error.

KNN: (K-Nearest Neighbors)

- KNN Classification: Classify to the majority vote of the K nearest neighbors in the training set.



k is the hyper parameter, it determines how wiggly our decision boundary is, which is the complexity of the model.

* $\left\{ \begin{array}{l} k=1 \text{ high complexity} \\ k=n \text{ low complexity} \end{array} \right\}$ *

KNN regression: takes the average of K nearest neighbors to minimize the MSE.

8/24/23 Lecture 2 MSE & Linear Regression, ESL 2.9-3.2 MLPP 7.1-7.3

Review: 2 Major objective

1st Goal: Find a function from training data to generalize well to future data.

supervised

→ Prediction of New data

(i) split

(ii) learn $\hat{f}$ from (x^{tr}, y^{tr})

(iii) Predict $\hat{y} = \hat{f}(x^{test})$

2nd Goal: Interpretation, Insight, Data driven discovery

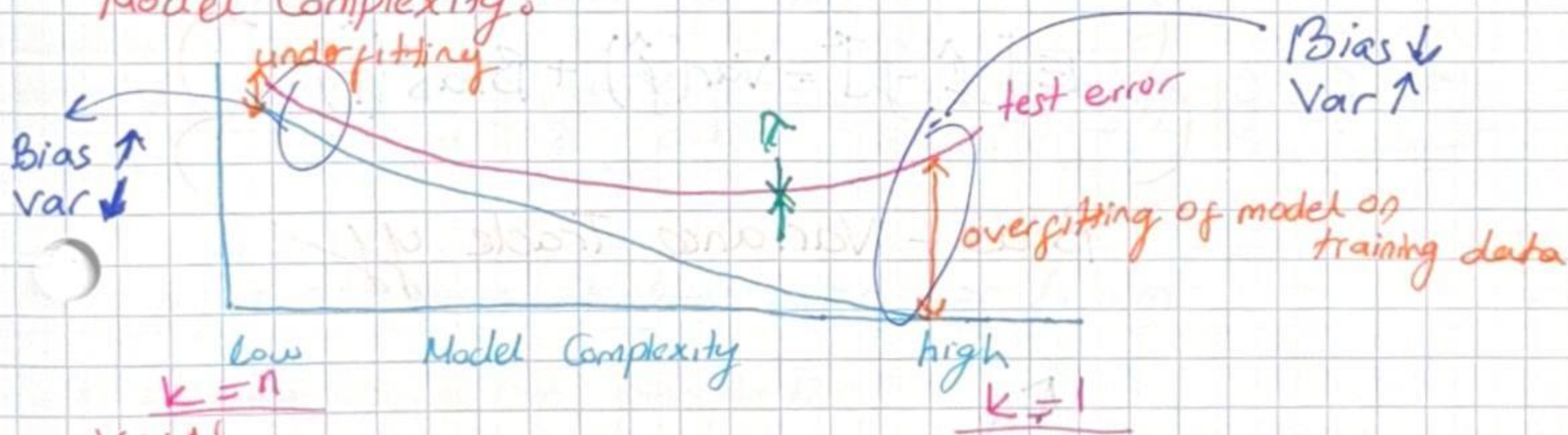
supervised & unsupervised

* All unsupervised data related to interpretation.

* For supervised; it is explanation of a prediction

* Which features are important for prediction.

Model Complexity:



if: $p > n$ high dimensional data; our matrix is overrepresented
alot of features unnecessary.

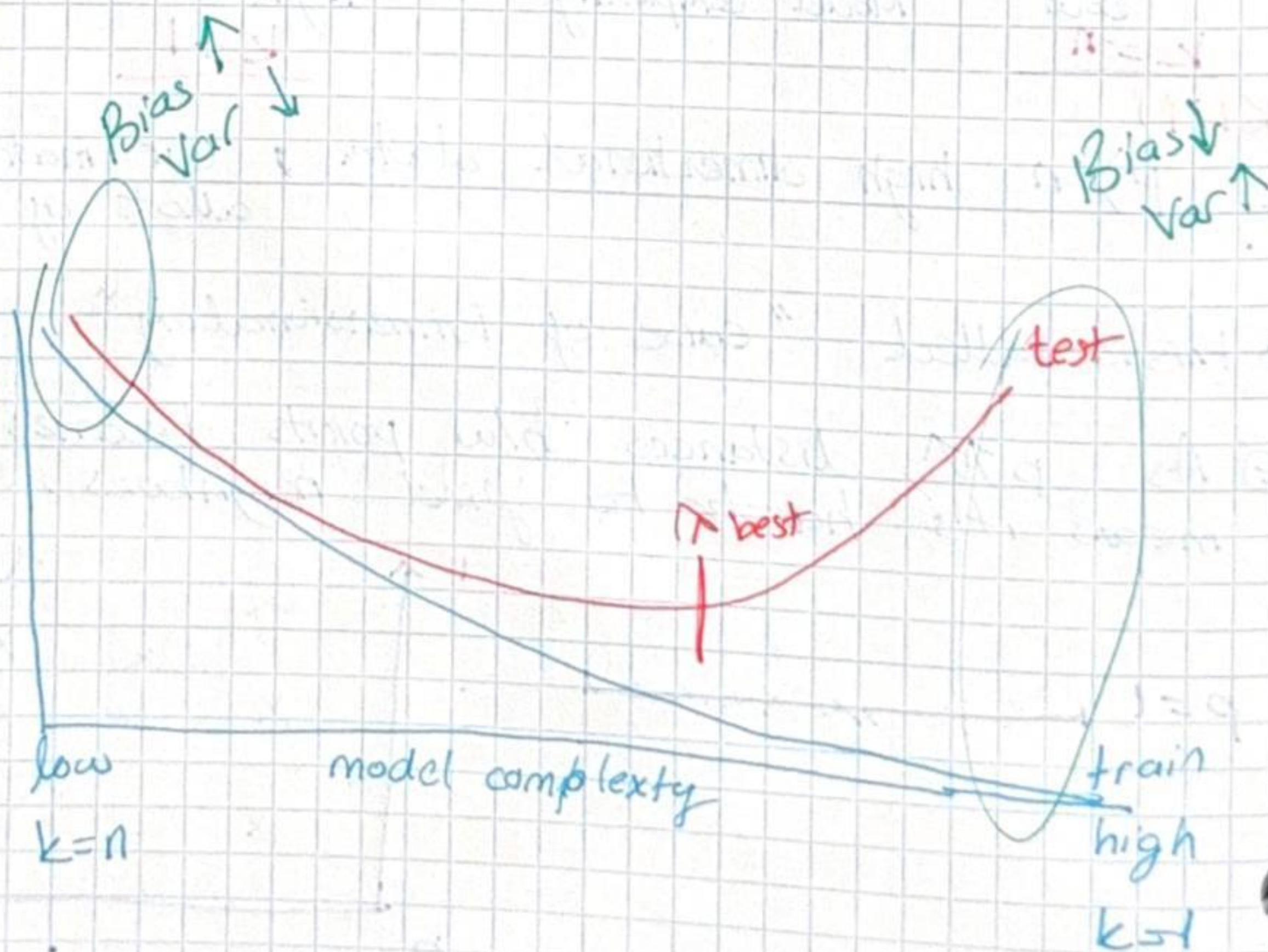
→ this is called "Curse of Dimensionality"

⊕ As $p \uparrow$, distances b/w points becomes larger, which means it's harder to find neighbors.



$$E[\hat{f} - f]^2 = \text{Var}(\hat{f}) + \text{Bias}^2(\hat{f})$$

Bias - Variance Trade off



INTRODUCTION to REGRESSION:

data (x_i, y_i) , $i = 1, 2, \dots, n$ $x_i \in \mathbb{R}^p$, $y_i \in \mathbb{R}$

Regression model $\Rightarrow y_i = f(x_i) + \epsilon$

* ϵ : distributed as independent noise, with mean 0, and σ^2 variance

$\epsilon \sim (0, \sigma^2)$ We assume we're giving data x, y pairs. where x are fixed, but our added noise is random.

Goal is:

(i) use our training data to get $\hat{f}$, $\rightarrow \hat{f}$

(ii) get $\hat{y}^* = \hat{f}(x^*)$

Error: how close y^* and $\hat{y}$

* 1. MSE: $\frac{1}{n^*} \|y^* - \hat{y}\|_2^2 = \frac{1}{n^*} \sum_{i=1}^n (y_i^* - \hat{y}_i)^2$ $\hat{y} = \hat{f}(x^*)$

2. MAE, mean absolute error $\rightarrow L_1$ norm

MSE from Population perspective:

$$E[\hat{f}(x^*) - f(x^*)]^2$$

$$= E[\hat{f} - f]^2$$

$$= E[(\hat{f} - E(\hat{f})) - (f - E(\hat{f}))]^2 - 2(\hat{f} - E(\hat{f}))(f - E(\hat{f}))]$$

$$= E[(\hat{f} - E(\hat{f}))^2] + E[(f - E(\hat{f}))^2]$$

$\text{Var}(\hat{f})$

$\text{Bias}^2(\hat{f})$



Bias $\uparrow$

Bias $\downarrow$, Var $\downarrow$

Bias $\downarrow$, Var $\uparrow$

$$* E[\hat{f} - f]^2 = \text{Var}(\hat{f}) + \text{Bias}^2(\hat{f})$$

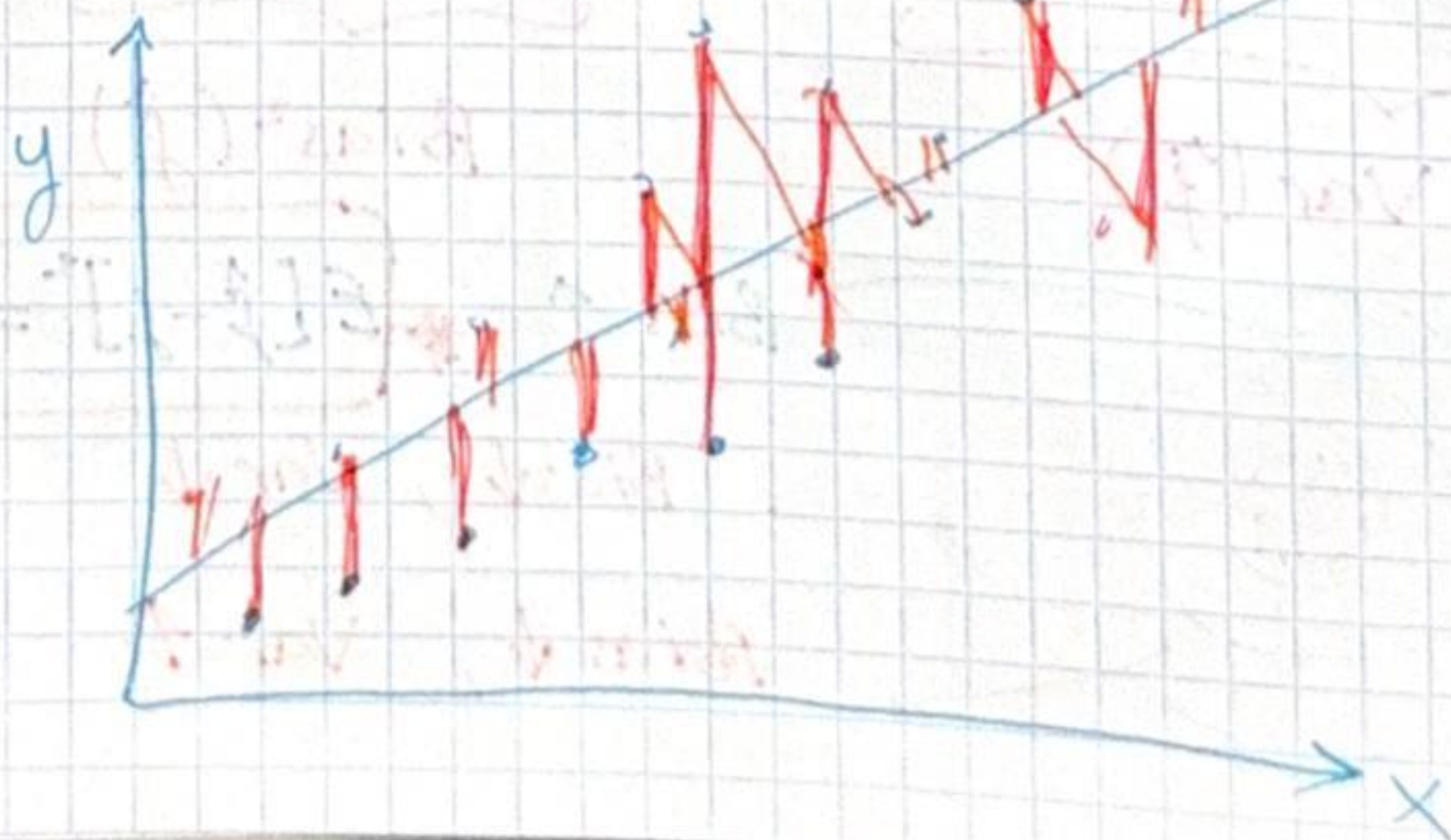
$\left\{ \begin{array}{l} \text{Bias} \\ \text{Var} \end{array} \right\}$
Trade off

$$\hat{y} = x \hat{\beta}$$

$$y = x \beta + \epsilon$$

Geometric Interpretation:

H: Projecting y onto x

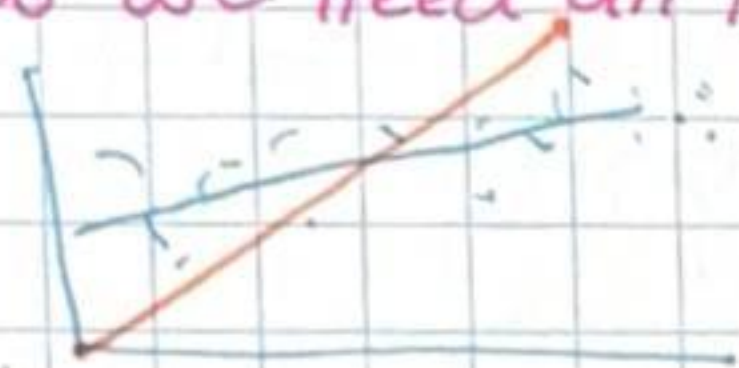


LINEAR REGRESSION

$$y_i = f(x_i) + \epsilon$$

$$f(x_i) = \beta_0 + x_i^T \beta$$

Do we need an intercept?



yes!

$\beta_0 \in \mathbb{R}$, intercept

$\beta \in \mathbb{R}^p$, slope of the hyperplane

$$y = X\beta + \epsilon \quad \text{model}$$

How can we avoid worrying about intercepts?

- 1) $\tilde{X} = \begin{bmatrix} 1 & \vdots & X \end{bmatrix}$ Add a 1's vector to the data.
- 2) Center the y , and columns of X .

Fitting Linear Regression:

* Empirical Risk Minimization: Solve an optimization problem to find the optimal parameters that minimize some error (loss) function.

$$\min_{\beta} \frac{1}{2} \|y - X\beta\|_2^2$$

$$\frac{\partial}{\partial \beta} = X^T (y - X\beta) = 0$$

$$X^T y = X^T X \beta \Rightarrow \hat{\beta} = (X^T X)^{-1} X^T y$$

$$\hat{y} = X \hat{\beta} = X (X^T X)^{-1} X^T y$$

H : "Hat matrix"

H is a projection matrix; which is idempotent

eigenvalues $\in \{0, 1\}$

H : projecting

y onto X axis

$$H = H^2$$

Statistical properties

① MSE is a function of bias and variance:

$$\begin{aligned} \textcircled{1} \quad E[\hat{\beta}] &= E\left((X^T X)^{-1} X^T y\right) \\ &= (X^T X)^{-1} X^T E(y) \end{aligned}$$

$$y = X\beta + \epsilon$$

$$E(y) = X\beta$$

$$E[\hat{\beta}] = \beta$$

NO BIAS!

* Out of all that class of linear models, least squares have no bias!

* So all of the MSE error comes from Variance!

$$\begin{aligned} \textcircled{2} \quad \text{Var}(\hat{\beta}) &= \text{Var}\left((X^T X)^{-1} X^T y\right) \\ &= X (X^T X)^{-1} \underbrace{\text{Var}(y)}_{\sigma^2} (X^T X)^{-1} X^T \end{aligned}$$

$$\text{Var}(\hat{\beta}) = (X^T X)^{-1} \sigma^2$$

$$\textcircled{3} \quad \epsilon \sim N(0, \sigma^2)$$

optimal $\rightarrow \hat{\beta} \rightarrow$ Gaussian MLE

$$\min_{\beta} - \mathcal{L}(y|X\beta)$$

$\hat{\beta} \rightarrow$ z-test $\uparrow$ (t-test)

f-test fit of $\hat{\beta}$

χ^2 -test for $\text{Var}(\hat{\beta})$ or resid.

$\beta^{ls} \Rightarrow$ least square bias = 0
variance is min compared to other models

$$(y^T X^T + (X^T X)) \beta = [A] b$$

$$(y) \equiv y^T X^T + (X^T X) =$$

$$y^T X^T + X^T X =$$

$$[A] = [b]$$

$$[A] = [b]$$

$$3 + 2X = 11$$

$$8X = 8$$

from Lab, cont.

p Value

* We cannot only get the features that are $p \ll 0.05$ because it is conditional.

$\hat{\beta}_j$ is conditioned to all other features in the model

If the features are independent, which means orthogonally then we can take them separately.

8/29/23 Lecture 3 - Ridge Regression ESL 3.2 & 3.4 MLPP 7.2-7.3, 7.5

Regression is a prediction problem with continuous response. → real value

Regression: $y = f(x) + \epsilon$

Linear regression: $y = \beta_0 + x^T \beta + \epsilon$

Solve this with some risk minimization; Loss function:

$$\Rightarrow \min_{\beta} \frac{1}{2} \|y - X\beta\|_2^2$$

Intercept was eliminated with adding a column of 1's

(2) centering columns of X and y . (subtract the mean)

$$\hat{\beta}^{ls} = (X^T X)^{-1} X^T y$$

with this equation finding the hyperplane to minimize all projection of y onto X or

$$E(\hat{\beta}^{ls}) = \beta \Rightarrow \text{unbiased}$$

$$\text{Var}(\hat{\beta}^{ls}) = (X^T X)^{-1} \sigma^2$$

$$\text{MSE} = \text{Var} + \text{Bias}^2$$

= 0

All MSE is coming from Variance because Bias = 0

Gauss - Markov Theorem: Out of all linear unbiased estimators of β , $\hat{\beta}^{ls}$ has the minimum variance.

* Lowest MSE (Best Linear Unbiased Est BLUE)

from Lab: We can only interpret magnitude of $\hat{\beta}$ iff the scale of the features is the same.

Need to standardize the cols of X . Subtract the mean and divide by variance.

When will Linear Regression fail?

(i) When the true $f(x)$ is not linear.

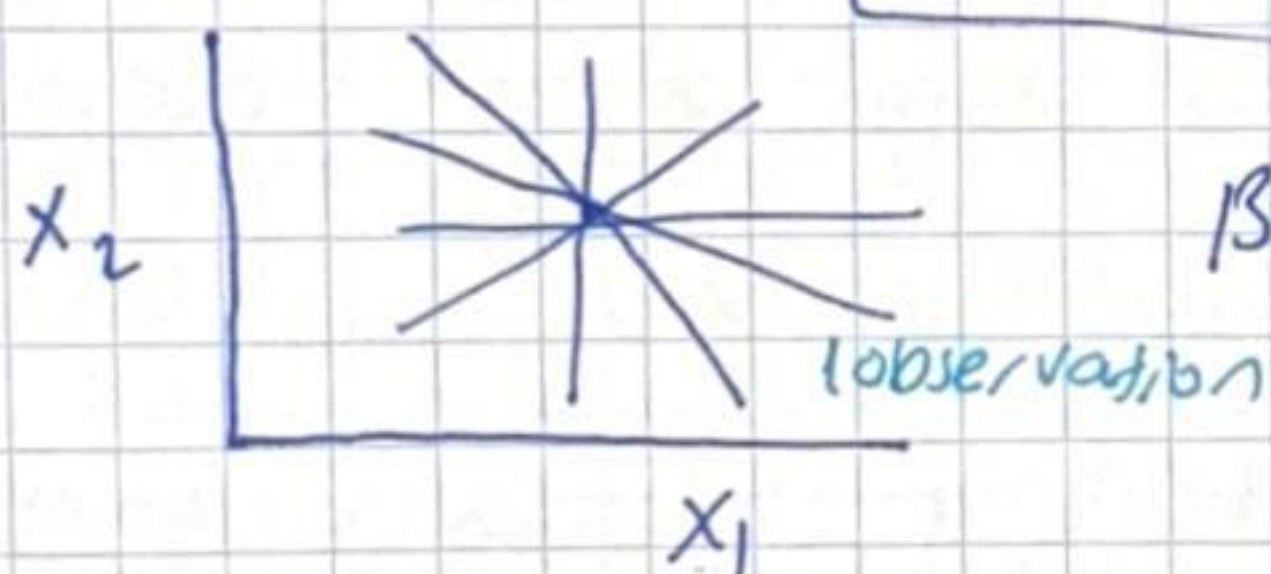
(ii) Anomalies, error

* (iii) If we are in a high dimensional setting

$p > n$ high dimensional

$$\hat{\beta}_{LS} = (X^T X)^{-1} X^T y$$

(~~does~~ not exist. Solution is not unique, that gives infinitely many solutions!)



Bias = 0, Var $\uparrow\uparrow\uparrow$
infinitely many solution

Claim: Training error = 0

Proof: $\hat{y} = H y$

$$X(X^T X)^{-1} X^T = H$$

number of e-vals = 1 is $\text{colrank}(X)$

$$\text{colrank}(X) = n$$

$H = I$!! which means $\hat{y} = y^{\text{train}} \Rightarrow \text{Error} = 0$ (Training)

How can we remedy this!

Limit features, reduce dimension of X , get more data.

add a constraint that "controls" the variance.

Idea: Constrain hyperplanes to bias then towards

certain solutions

increase Bias $\uparrow$, hoping Var $\downarrow\downarrow$

regularization

Regularization:

these two optimization problems exist

$$\textcircled{1} \min_{\beta} \frac{1}{2} \|y - X\beta\|_2^2$$

if $p()$ is convex

Add a constraint, $p(\beta) \leq \tau$

p is a penalty function, or regularization function, that maps p back to real.

$$\textcircled{2} \min_{\beta} \frac{1}{2} \|y - X\beta\|_2^2 + \frac{\lambda}{2} p(\beta)$$

$$p(): \mathbb{R}^p \rightarrow \mathbb{R}^+$$

1-1 mapping $\lambda \leftrightarrow \tau \rightarrow \text{Lagrange}$

$\lambda \geq 0$; is a regularization (penalty) parameter, hyperparameter

There exists a λ that makes these optimization problems equal.

What type of $p(\beta)$ do we want?

In $p \gg n$, all features are super correlated. with each other - they are not independent. They appear correlated because we don't have enough data.

Assume j and k are super correlated, which gives $\beta_j \uparrow \uparrow$. I can't send $\beta_k \downarrow \downarrow$ down

We form $p(\beta) = \|\beta\|_2^2$
Ridge $\|\beta\|_2^2 = \sum_{j=1}^p \beta_j^2$

Ridge Regularization? (Tikhonov)

$$\textcircled{*} \min_{\beta} \frac{1}{2} \|y - X\beta\|_2^2 + \frac{\lambda}{2} \|\beta\|_2^2 \longrightarrow \hat{\beta}^r = \frac{\hat{\beta}^{ls}}{1 - \lambda}$$

ridge regression optimization

$\lambda \geq 0$ hyperparameter (controls Model Complexity)

$\lambda \rightarrow 0$ least squares

$\lambda \rightarrow \infty$ $\|\beta\|_2^2 \rightarrow 0$, β 's $\rightarrow 0$ (approach 0)
constraint β to 0.

Ridge solution:

$$\frac{\partial}{\partial \beta} = -X^T(Y - X\beta) + \lambda\beta = 0$$

positive definite matrix

$$\hat{\beta}_r = (X^T X + \lambda I)^{-1} X^T Y$$

* This solution exists always, and unique! even when $p > n$ ***

Interpretation: we didn't add intercept here.
we do center the columns of X (Y)

* Standardization: If features are on a different scale, it is difficult to interpret parameters w/ regularization we need to standardize features.

$$E(\hat{\beta}_r)$$

Expectation: Bias

$$E[\hat{\beta}_r] = [(X^T X + \lambda I)^{-1} X^T X] \beta$$

$$E[\hat{\beta}_r] = -\lambda (X^T X + \lambda I)^{-1} \beta$$

biased toward zero

$$\text{Var}(\hat{\beta}_r) = \sigma^2 (X^T X + \lambda I)^{-1} X^T X (X^T X + \lambda I)^{-1}$$

property

$$\text{Var}(\hat{\beta}_r) \leq \text{Var}(\hat{\beta}_{ls}) \quad \forall \lambda > 0$$

MSE existence theorem:

$$\exists \lambda > 0 \text{ such that } \text{MSE}(\hat{\beta}_r) < \text{MSE}(\hat{\beta}_{ls})$$

Ridge always wins!

* Ridge is great for prediction
* Ridge penalties $\|\beta\|_2^2$ appear all over ML. (kernel, svm, ... GMM)

Review Regularization: Idea of regularization is to add a little bias for hopefully big reduction in variance

Constraint: $P(\beta) \leq \tau$; $P(): \mathbb{R}^r \rightarrow \mathbb{R}^+$

$$\min_{\beta} \frac{1}{2} \|y - X\beta\|_2^2 + \frac{\lambda}{2} P(\beta)$$

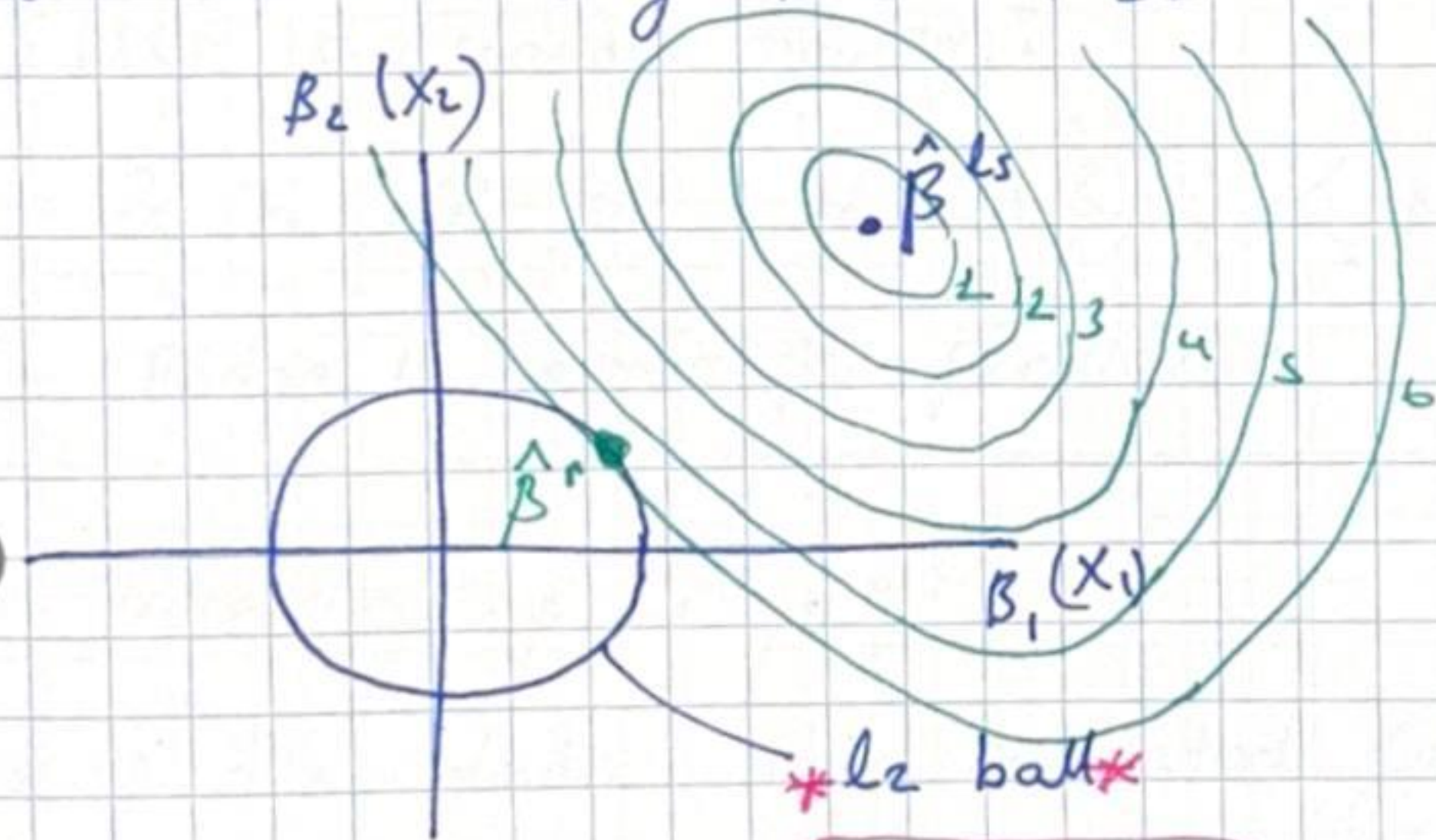
In general regularized problem

$$\lambda \geq 0$$

hyper parameter / regularization p / penalty / tuning

λ , hyperparameter modulating the trade off for how well

we are minimizing the MSE vs the constraint and penalty.



In Ridge regression we use penalty of two norm of β

$$\min_{\beta} \frac{1}{2} \|y - X\beta\|_2^2 + \frac{\lambda}{2} \|\beta\|_2^2$$

$$\|\beta\|_2^2 \leq \tau$$

Least square solution is at β^{ls} . The dot is when MSE is = 0, for each line it is equal to 1, 2, 3, ...

Ridge solution is when these contours hits the ball of constraints. It pulled the least square solution, β^{ls} ,

to zero. λ penalty is modulating the trade of between

how well we fit our penalty, risk minimizer, loss function, and how close we're pulling those β towards zero.

Ridge Solution:

$$\hat{\beta}^r = (X^T X + \lambda I)^{-1} X^T y$$

* unique

* always exists

* When using regularization, we need to standardize the features, so λ affects all features equally.

Bias & Variance of Ridge Solution:

MSE Existence theorem:

$$\exists \lambda > 0 \text{ st } \text{MSE}(\hat{\beta}_\lambda^r) < \text{MSE}(\hat{\beta}^u)$$

Ridge is better for prediction !

Properties of Ridge:

* If the data has correlated features; for ridge,

if $\text{cor}(x_k, x_j) \uparrow$, then; $\hat{\beta}_j^r \rightarrow \hat{\beta}_k^r$

it pulls the β 's together. If the correlation is one

then the two β will become same.

* Ridge performs well with correlated features.

Correlation means poor performance at least squares

but, it helps Ridge regression. Means actually

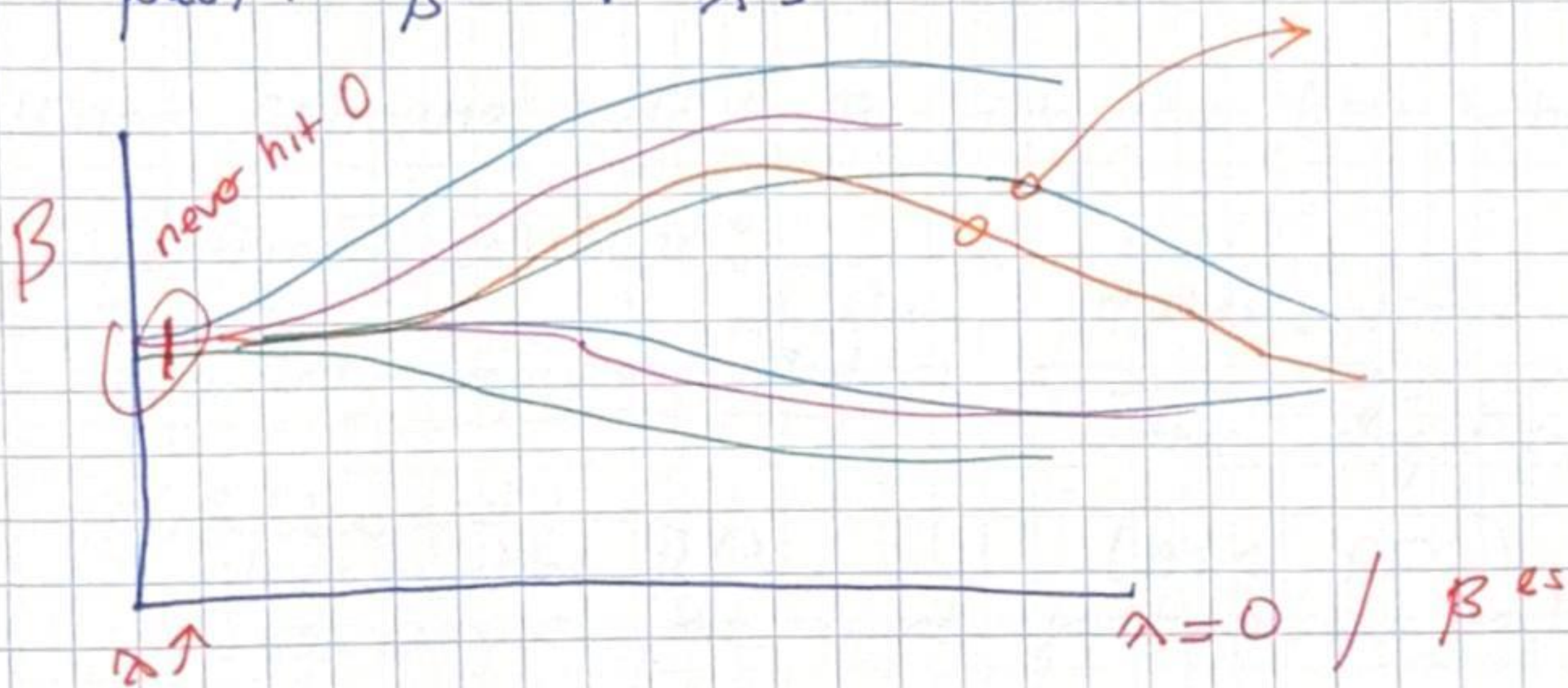
Ridge performs better than others with correlated features.

Auto-denoising?
* Ridge shrinks larger patterns in X less than smaller patterns. It is doing Regression on denoised X

Ridge shrinks the noisy part of X more than the unnoisy part of X . $\Rightarrow$ called auto de-noising

Regularization Paths:

* compute $\hat{\beta}_\lambda$, $\lambda = 0, 1, \dots, \lambda \uparrow$
plot. $\hat{\beta}_\lambda \forall \lambda$'s



Select Features:

$$k < n, \quad k \ll p$$

If $p > 0$, we select a subset of features to use.

(1) Feature filtering? Ex: remove features with $\text{cor}(x_j, y) \downarrow$
It refers to looking at marginal associations. It means looking at one feature at a time.
 $\hookrightarrow$ univariate features.
This is not always the best for multivariate models.

(2) Feature selection:

Find the best subset for multivariate models

BEST SUBSETS

$$\min_p \frac{1}{2} \|y - X\beta\|_2^2$$

we minimize the empirical risk but we add a constraint that ~~the~~ ^{the} of

$$\# \beta_j \text{'s (non-zero)} \leq k$$

"L0 norm"

$$k \leq n$$

$$k \leq p$$

(*) Non convex problem

combinatorially hard

$$\|\beta_0\| \leq k$$

$$= \sum_{j=1}^p \mathbb{I}(\beta_j \neq 0)$$

$$\binom{p}{k}$$

What can we do? To pick the Best Subsets?

(1) Algorithmic solutions:

(i) Greedy stepwise solution:

algorithm: makes optimal decisions at each stage w/o taking other stages in account.

(ii) Backward:

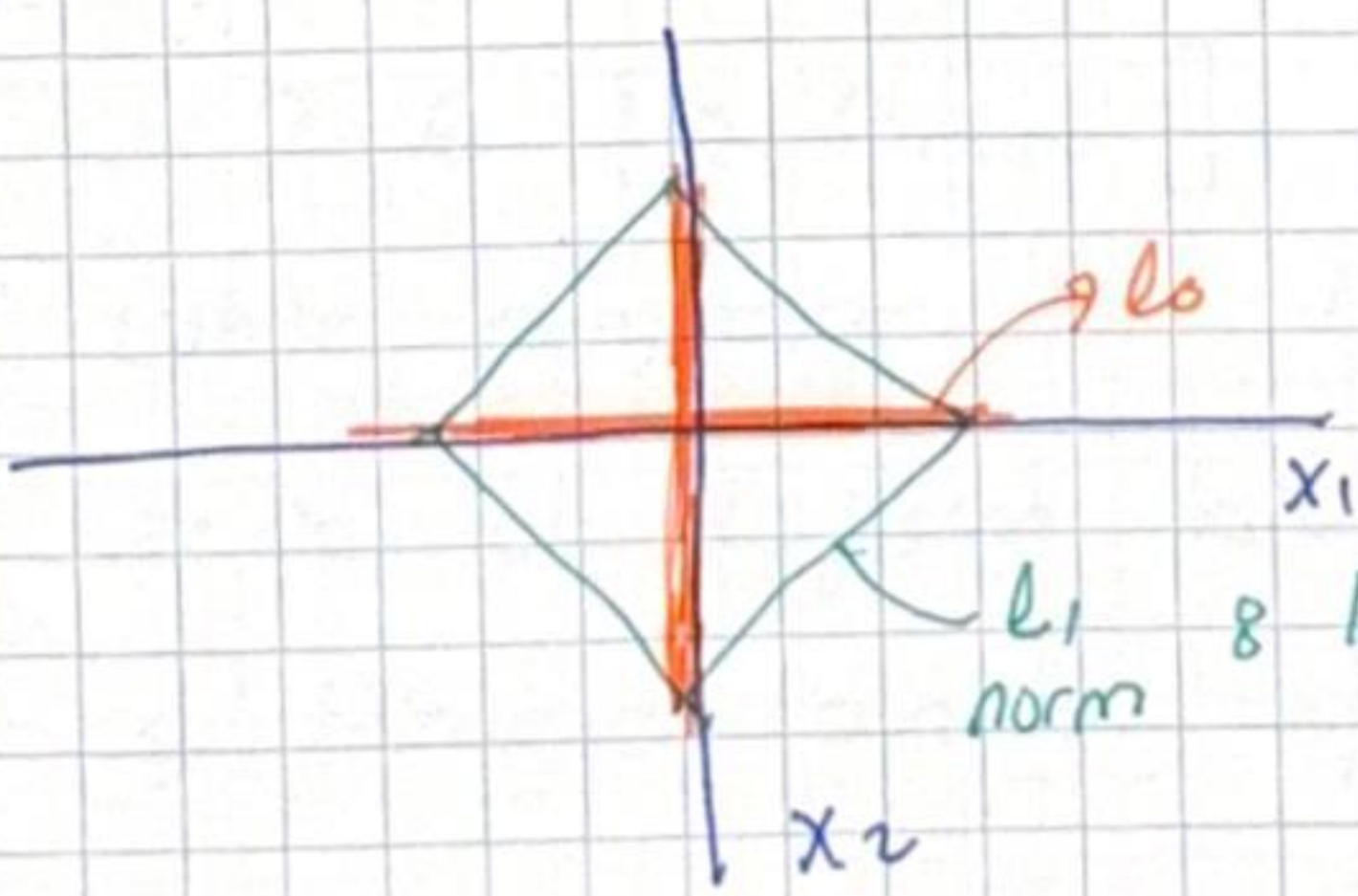
start with β and remove the worst features at each step. (AIC, BIC)

(iii) Recursive feature Elimination (RFE)

removes r features at each step.

(iv) Forward:

(2) Best convex relaxation of the (L0-norm)



Best convex relaxation of β_0

$$\|\beta\|_1 \leq \tau = \sum_{j=1}^p |\beta_j|$$

LASSO

From labs:

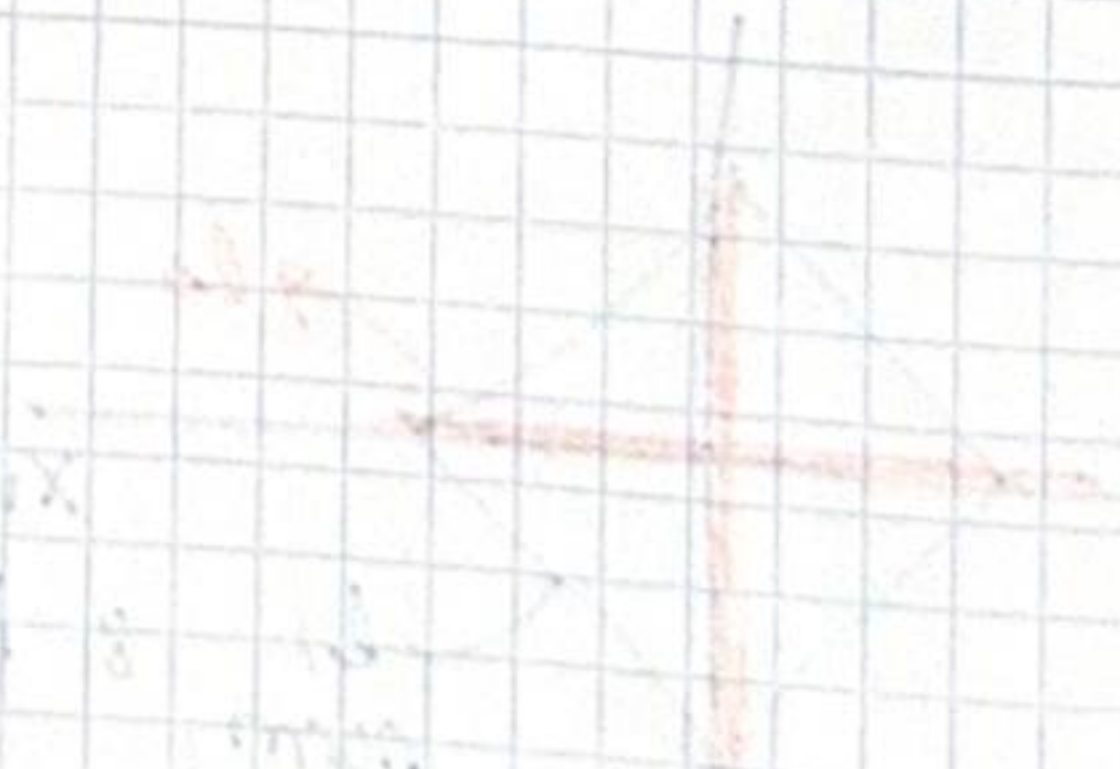
It is good-is for prediction

It is good for truly sparse model

* Good for interpreting important features

parsimony : reduced model complexity

* in general ridge is better



LASSO :

Solve ^{empirical} ~~minimum~~ risk minimization.

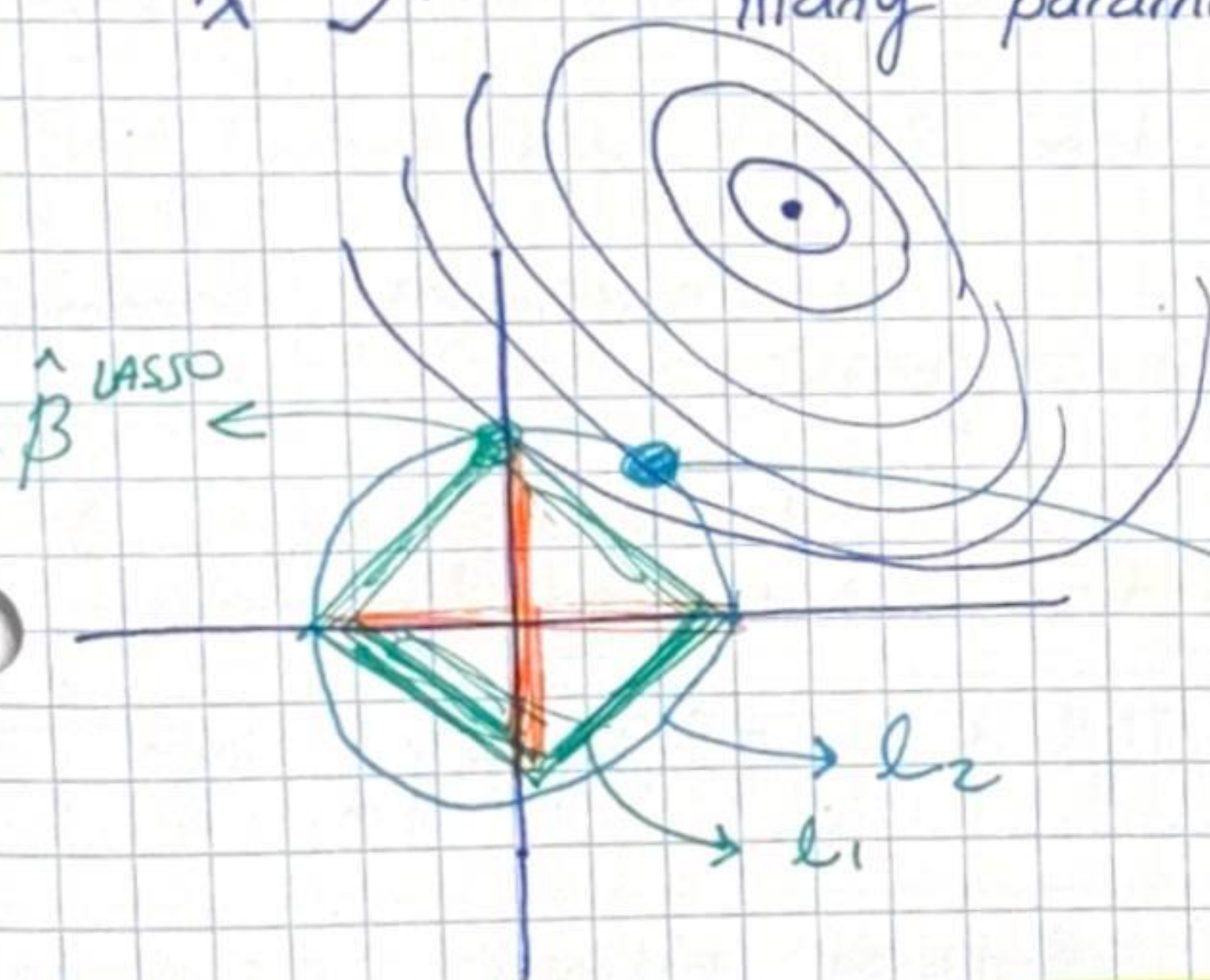
$$\min_{\beta} \frac{1}{2} \|y - X\beta\|_2^2 + \lambda \|\beta\|_1$$

Lasso optimization problem

$\lambda \geq 0$ tuning parameter

$\lambda \rightarrow 0$ least square solution

$\lambda \uparrow$ many parameters will be $= 0$ with lasso
 β_j 's $= 0$



β^{LASSO} , gets the x, β_1 equal to zero

β^{LS} didn't get it pulled to zero, it shrunk β_1 a lot not equal to zero.

so $\Rightarrow$ lasso leads to **sparse solutions** **

(*) There always exists a value of λ such that $\hat{\beta}^L \equiv 0$,
we call this λ_{max}

$\exists$ a value of λ s.t. $\hat{\beta}_\lambda^L \equiv 0 \Rightarrow \lambda_{max}$

$$\lambda \in [0, \lambda_{max}]$$

(*) While computing the Lasso solution, there is

$\rightarrow$ no closed form solution, so we do

$\rightarrow$ Iterative convex optimization

Review Feature Selection vs filtering

In feature selection, this is a multivariate problem, considering all p features, while filtering is univariate, we only consider the marginal association between X_j & y .

Feature selection : multivariate : consider all p features

Feature filtering : univariate : marginal association of X_j, y

- 1) **Best subset**: Find the k best features, " l_0 "-norm, $\rightarrow$ non-convex hard to compute

Approaches : Algorithmic :

Greedy $\rightarrow$ Forward & Backward Stepwise

l_1 -norm regression - Lasso?

Idea: " l_1 "-norm is the best convex relaxation of " l_0 "-norm

$$\min_{\beta} \frac{1}{2} \|y - X\beta\|_2^2 + \lambda \|\beta\|_1$$

$\lambda \geq 0$ penalty parameter

$\lambda \rightarrow 0$ $\beta \rightarrow \hat{\beta}^{LS}$

$\lambda \uparrow$ $\hat{\beta}$ becomes sparse, (many $\hat{\beta}_j$ becomes = zero) (*)

$\lambda \uparrow \uparrow$ all $\hat{\beta}$ becomes zero ($\hat{\beta} \equiv 0, \lambda = \lambda_{\max}$)

SOLVING the LASSO?

l_1 -norm is non-differentiable, we will do via some

iterative convex opt solvers

Solving Lasso

$$\min_{\beta} \|y - \beta\|_2^2 + \lambda \|\beta\|_1$$

differentiate this

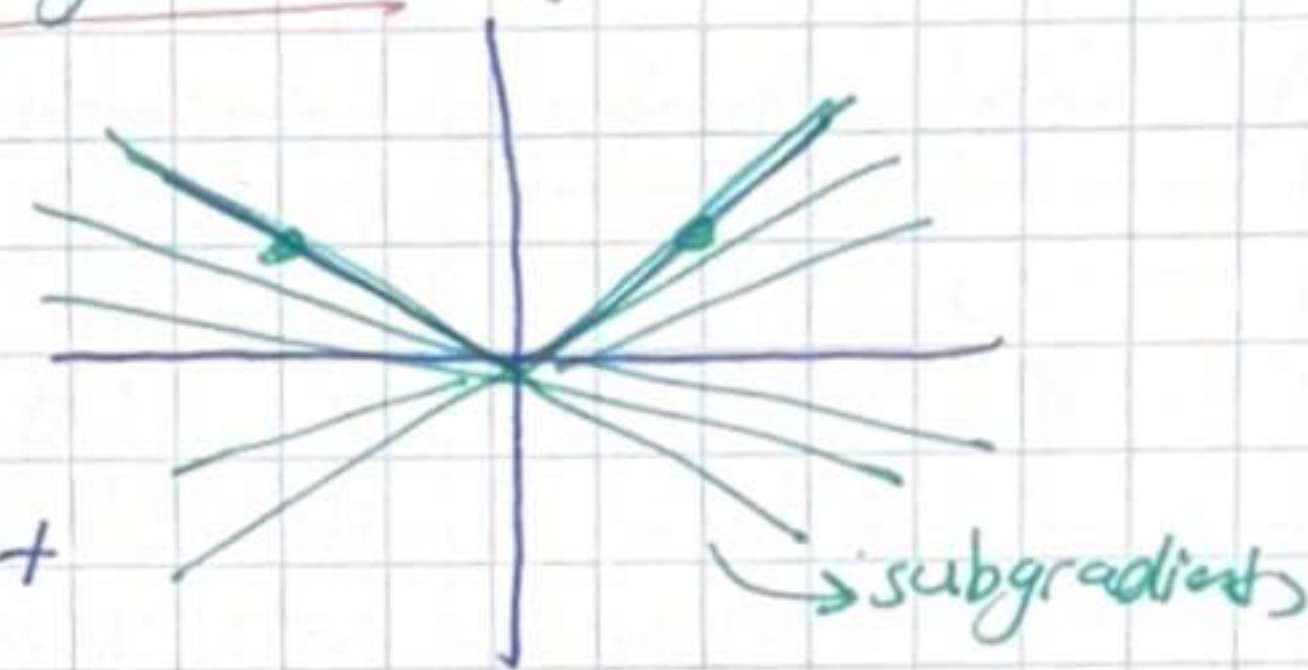
$$\frac{\partial}{\partial \beta} = -y + \beta + \lambda \Gamma(\beta) = 0$$

subgradient of β

$$\Gamma(\beta) = \begin{cases} \text{sign}(\beta), & \beta \neq 0 \\ [-1, 1], & \beta = 0 \end{cases}$$

either or $\times$
range

solve subgradient



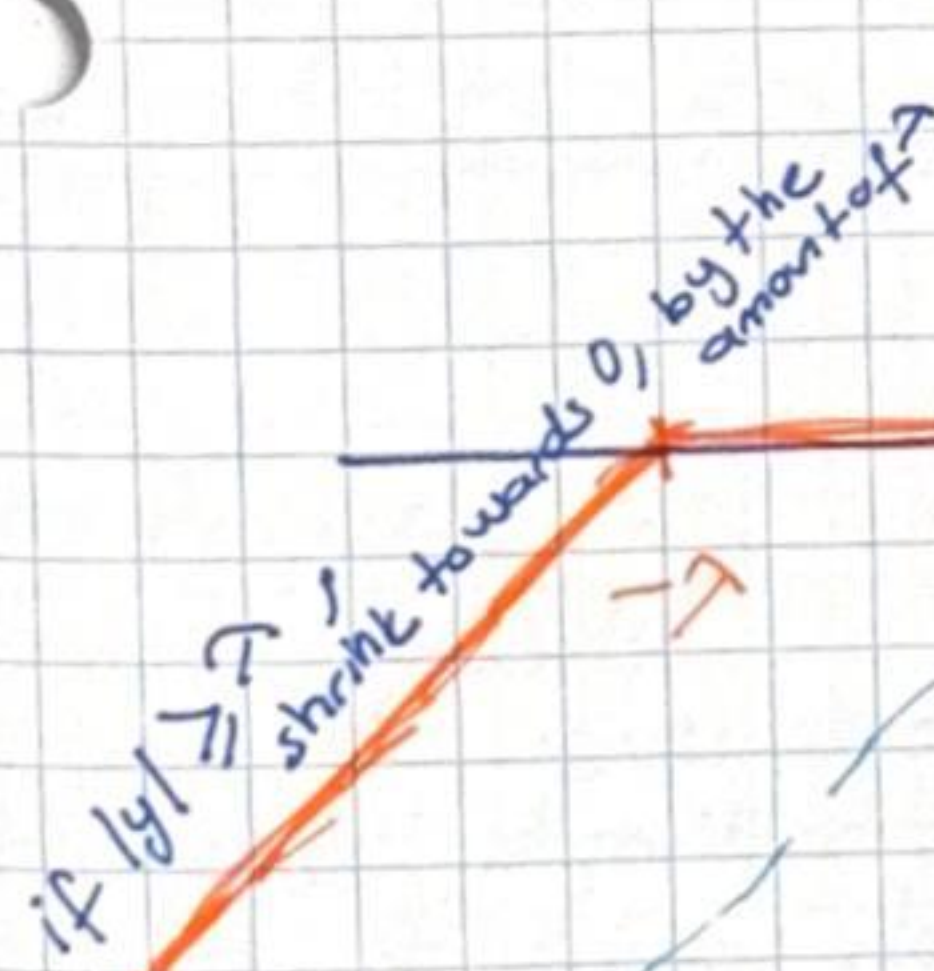
Solution:

$$\hat{\beta} = s(y, \lambda) = \text{sign}(y) (|y| - \lambda) +$$

Solve subgradient
soft thresholding function

No $\times$ here to demonstrate

$$= \begin{cases} 0, & |y| \leq \lambda \\ y - \lambda, & y > \lambda \\ y + \lambda, & y < -\lambda \end{cases}$$



$s(y, \lambda)$
soft thresholding function

⊗ If X is orthogonal,

$$\hat{\beta}^L = s(\hat{\beta}^O, \lambda)$$

LASSO :

$$\min_{\beta} \frac{1}{2} \|y - X\beta\|_2^2 + \lambda \|\beta\|_1$$

Solvers of optimization problem:

- i) Coord Descent
- ii) ADMM
- iii) Proximal Gradient Descent

Proximal Gradient Descent (Projected Gradient Descent)

Idea: Go down hill, (gradient) on the differentiable part, and project back onto the constraints of the non-diff part.

$$\min_{\beta} \underbrace{L(\beta)}_{\text{differentiable}} + \underbrace{\lambda P(\beta)}_{\text{non differentiable}}$$

$$\beta^{(t+1)} = \underbrace{\text{prox}_{\lambda P, \eta}}_{\text{projects onto}} \left(\underbrace{\beta^{(t)} - \eta \nabla L(\beta^{(t)})}_{\text{gradient Descent}} \right)$$

LASSO:

$$\beta^{(t+1)} = S \left(\underbrace{\beta^{(t)} + \frac{1}{L} X^T (y - X \beta^{(t)})}_{\text{Gradient Descent}}, \underbrace{\lambda/L}_{\text{step size}} \right)$$

extra details

$$L = \lambda \max(X^T X)$$

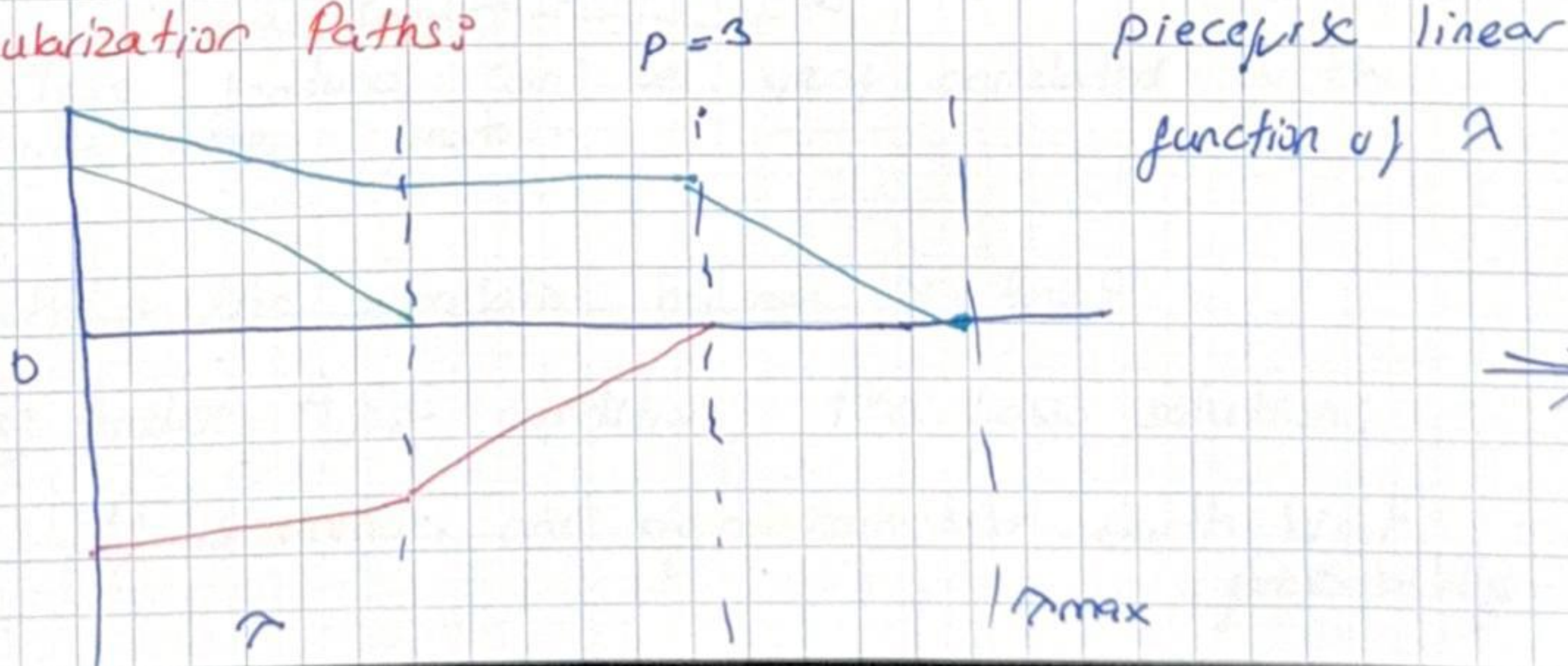
max e-val

Properties of LASSO solution:

(1) sparsity

- * It gives sparsity, its useful for interpretation to see the most important feature
- * It gives decreased model complexity; parsimony

Regularization Paths:



As we reduce $\lambda_{\max}$, most important feature (blue) comes. The direction of its path changes when another feature gets in the data. That happens when the features are correlated. That causes the slope change.

What is the effect of correlation on Lasso?
How does it affect interpretation?

Even with correlation, predictions are still good.
* Ridge is usually better.

Selection Consistency?

When probabilistically does the Lasso select the right feature?

Suppose the true features are X_S , non-true X_{S^c}

A) $\lambda_{\min}(X_S^T X_S) > \epsilon > 0$, then

* $\hookrightarrow$ true feature can't be too correlated

B) $\beta_{\min}$ condition, (signal strength) if too small you cannot represent

C) Irrepresentable (Incoherence) condition

$$\|(X_S^T X_S)^{-1} X_S^T X_{S^c}\|_{\infty} \leq 1 - \gamma \quad \gamma > 0.$$

* True features can't be very correlated with non-true features.

$\hookrightarrow$ this is like correlation between X_S & X_{S^c}

* Under A, B, C condition, the Lasso solution;

β_X^L is unique and sign consistent with high probability.

* In general LASSO does not select correct features with correlated data.

⇒ Interpret with caution!

Applications of Lasso in ECE:

1) Wavelet Denoising

Denoising a signal

$\min_{\beta}$

$$\|Y - W\beta\|_2^2 + \lambda \|\beta\|_1$$

→ wavelet basis matrix

$$\Rightarrow \hat{\beta} = S(W^T Y, \lambda)$$

orthogonal

2) Compressed Sensing: solution is Lasso. We get as little as possible data, to get the big picture. Like in MRI, $p \gg n$ (imaging)

Extensions of LASSO (other types of Regularization)

$$\min_{\beta} \frac{1}{2} \|Y - X\beta\|_2^2 + \lambda P(\beta)$$

1) Elastic Net:

→ lasso

→ ridge

$$P(\beta) = \alpha \|\beta\|_1 + (1-\alpha) \|\beta\|_2^2$$

$$\alpha \in [0, 1] \quad (\alpha = 0, \text{Ridge}) \quad (\alpha = 1, \text{Lasso})$$

* improves on Lasso when correlated features.

2) Fused Lasso / Total Variation Penalty

$$P(\beta) = \sum_{j=1}^{p-1} |\beta_{j+1} - \beta_j|$$

It will shrink all adjacent parameters to zero.
For data with spatially aligned



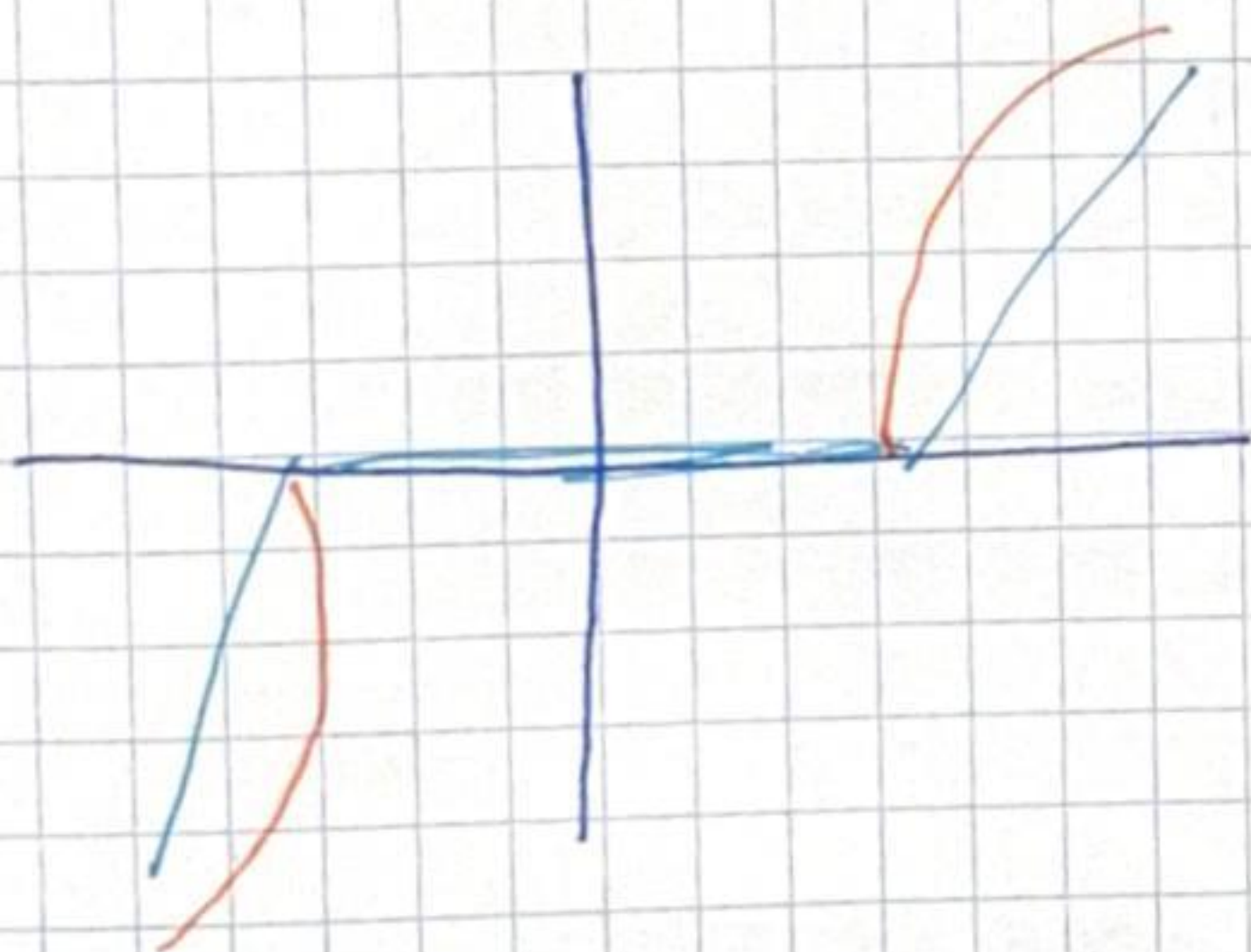
It is fusing the adjacent β together. Only at two point it is not zero.

Extensions for time series, spatial, signal

3) Non-convex Penalties

Merge between ℓ_0 and ℓ_1

SCAD, MCP



4) Group Lasso; select $\rightarrow$ pre defined group features

5) Adaptive Lasso

$\rightarrow$ Weighted Lasso

$\rightarrow$ Lasso is biased downwards

9/17/23 Lecture 6 Validation & Hyperparameter Tuning

ESL 7.10 MLPP 6.5

- * Goal of ML is to generalize to new data
- * select the best hyperparameters $\Rightarrow$ hyperparameter tuning!
- * Validate the best Model Family (e.g. Lasso vs Ridge)

How to Validate

(i) Data-Splitting

n observation; $\rightarrow$ split the data randomly split the observations



Why can't we use test set for hyperparameter tuning?
b/c we then peek at the test data.

- * Test data only seen & used once !!! that is only for prediction error.

(ii) Process of ML

ai) split the data $\rightarrow T_r, Val, T_s$

a ii) On training, for $\lambda = 1, \dots, \lambda_{max}$
fit Lasso $w / \lambda = \hat{\beta}_{\lambda_k} \Rightarrow \hat{\beta}_{\lambda_k}$

a iii) On Validation set, make Prediction $\hat{y}_k = X \hat{\beta}_{\lambda_k}$ for $k=1, \dots, K$

Select K that minimizes the $MSE(\hat{y}_k, y^{val})$

a iv) On test set, Evaluate Prediction MSE

$MSE(\hat{y}_{\lambda}, y^{test})$

Sometimes you can use training data to retrain, w λ^* or $[T_r, Val]$

How can we choose b/w 2 model families? Lasso/Ridge
e.g.

* Can't use Validation for both tuning & selecting model families
↳ This overfit to validation

* Make Query set to choose best model family.

* Rule of thumb: If you need new unseen data for

- i) new tuning
- ii) selecting best model
- iii) evaluate prediction error.

How do we validate with limited data?

Cross Validation: Use random folds of the training set as validation set.

K-fold cross validation

	1	2	3	4	
1	tr	tr	tr	✓	
2	tr	tr	✓	tr	
3	tr	✓	tr	tr	
4	✓	tr	tr	tr	
	test				

Randomly split the training data into k folds: ($k=4$)
 $k=4$ 4 different model fits

exm: for $k=1 \dots k$ steps (fold)
Train on $-k$ chunk
Validate on $+k^{\text{th}}$ chunk

↳ get prediction error
End

C validation error =

Average Pred. Err.
over the k folds

CV typically used for hyperp. tuning.

Rule: Min. Rule $\lambda^* = \underset{\lambda}{\operatorname{argmin}} C_{\text{ver}}$

1 SE rule: $\lambda^{*SE} = \underset{\lambda}{\operatorname{argmax}} C_{\text{ver}, \lambda} < \overline{C_{\text{ver}, \lambda^*}} + SE \overline{C_{\text{ver}, \lambda^*}}$

Idea: sometimes the CV. curve can be flat, so choose the least complex method that is within 1 SE minimum

Notes on Cross validation:

1) Always we use unseen data for validation (choosing λ)
Validation chunk is unseen in the k th step

2) CV is typically used to choose λ
Need to use all training data to retrain at λ^*

3) Which k ?

if $k \downarrow$; tr, val $\rightarrow$ not enough data for training.

if $k \uparrow$; leave one out CV LOOCV, generalized CV
($k=n$)

usually used $k=5$
 $k=10$

issues: computational

: overfitting, haven't changed training / val data enough
overfitting with tabular data

not problem for sequential data

Can CV error be used as an estimate of prediction error?

No!

i) "seen" all data

ii) CV error used less data for training, higher than true prediction error.

Can CV be used to select the best model family? No

Instead

tr	tr	v
v	tr	tr
tr	v	tr
tr	tr	v

Discussion on CV

$n=300$, $p=30,000$

wrong place

I filtered features to top 1000, (corr XY)
I split the data

On tr.

$\rightarrow$ CV to select λ^*

$\rightarrow$ retrain on Tr at λ^*

- Pred. error from test

ESL 5.8 & MLPP 15.2 & 15.4
9/12/23 - Lecture 7 - Validation & Introduction to Non-Linear Regression.

Review Validation: Why? important? $\Rightarrow$ Goal? Generalizability

Goal - Purpose -

Select the best model for generalizability.

For every time we need to pre-evaluate a prediction error we need a new data.

* Large $n \rightarrow$ split $\rightarrow$ training / Validation / Query / Test

* ~~Large~~ $n \rightarrow$ cross validation (CV) $\rightarrow$ K-fold; train in $k-1$ fold, predict at k -th data.

Hyperparameter tuning: Search

$\hookrightarrow$ Grid search

- bad strategy for multiple tuning parameters. It will make the computation last longer, for A, α for example.

$\hookrightarrow$ Random search

$\hookrightarrow$ Binary search. Pick two point, then check 2 more in bw...

$\hookrightarrow$ Bayesian optimization: sequential

* model based optimization.

Start with random and search and build a predictive model for the prediction error, as a function of the hyperparameters.

Suggest next best, optimal for hyperparameter combos.

You do iterative search.

Idea of Validation: Interpretation.

feature selection

Feature Selection

(vs) Prediction

For instance, we use LASSO to select features, we care more about getting the best features than good prediction.

For interpretation of best features, can we still do same steps that we do for prediction?

Predictability: $T_r, V_L, T_t \Rightarrow$ model (select hyperparameters that minimize prediction error)

Can we use the same λ^* that minimize the validation prediction error the same that is best for feature selection interpretation.

No! minimizing prediction Error overselects features. More features almost always better for prediction! what can we do

Validation for Interpretability: selecting features

1) Stability: True interpretations (interpretations that generalize well) are stable with respect to random data perturbations.

The idea is to prevent overfitting to training data.

* We want to find true interpretations, that generalize well, and are stable with respect to random data perturbations.

Approach:

For $b = 1 \dots B$, you create X^* which is a randomly perturbed version of X^{tr} , of our training data.

- Exm: ① Take Random subsets from our training data set (Randomly perturb it)
- ② We can do bootstrapping; which is random subset sample with replacement.
 - ③ We can add random noise or random corruptions.

This is like making a new data set, sampling a new training data set.

Do this for X^* b times.

Fit our ML model on $X^{*,b}$ & record the presence of each interpretation. Which means, record/save which features were in or not.

Exm: Fit LASSO at this lambda λ^{*cv} , and record which features are selected, or not.
 weather that

* Stable interpretations are those with the highest prevalence; that are selected most often.

* The idea is; stability analysis is to report the stable interpretations. It is very hard to overfit to a training set, if we randomly trained different training sets, and only record those that are stable, occur often in different training sets.

* Stability is best for thus to prevent overfitting *

2) Predictability: (data splitting)

Idea: If you found a true interpretation, we should be able to predict the ~~the~~ interpretation on new data.

Exm: LASSO

- min40
- i) Rank order ^{selected by} (the Lasso) the top $k=1 \dots K$ features; most stable, best coefficients, ... etc.
 - ii) Fit a new model to x^{tr} only using $k=1 \dots K$ best features & record the test Prediction error.

NON LINEAR REGRESSION

non linear function

$$y_i = f(X_i) + \epsilon \quad \text{noise } \sim (0, \sigma^2)$$

Basis Expansion:

$$\tilde{X} = \begin{bmatrix} x_1 & x_1^2 & \dots & x_1^d & x_1 x_2 & \dots & x_1 x_2 x_3 & \dots \end{bmatrix}$$

not the way of doing this $\nearrow$ PTM, computation

Major Non-Linear Methods $\rightarrow$ Splines $\rightarrow$ not talking in class $*$
 $\rightarrow$ Kernels

* Kernels extends from basis expansion. Idea is to have an infinite dimensional basis expansion.

$$f(x) = \sum_{i=1}^{\infty} c_i \phi_i(x)$$

basis function $\phi_i(x)$
 * infinitely many basis expansion.
 coeff. function c_i

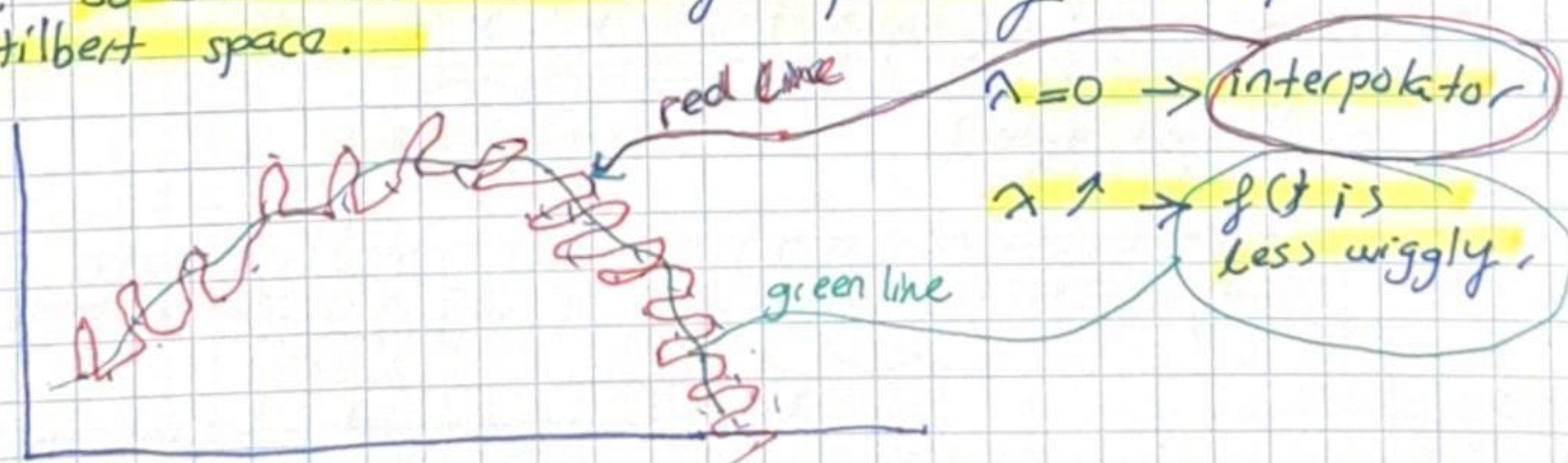
* * * If I had infinitely many basis expansions, I could fit any non linear function possible.

$f \in H_K$: reproducing Kernel Hilbert space

$$\min_{f \in H_K} \|y - f(x)\|_2^2 + \lambda \|f\|_{H_K}^2$$

Ridge for RKHS
 penalizes the curvature

If we have infinitely many functions and we are minimizing the MSE, (since f is infinite) we will have mass of overfitting. So we add a ridge penalty to reproduce kernel Hilbert space.



Mercer's Representer Theorem: Even if $f()$ is infinite-dimensional, we only need to evaluate $f()$ at the n training points.

It takes infinite dimensional computation down to n -dim computation.

exm: Kernel Ridge Regression

$$f(x) = \phi(x)^T \beta \quad \text{basis expansion}$$

$$\min_{\beta} \|y - \phi(x) \beta\|_2^2 + \lambda \beta^T \beta$$

p dim $\beta = \phi(x)^T \alpha = n$ dim unknowable, no idea what α could be

$$\min_{\alpha} \|y - \phi(x) \phi(x)^T \alpha\|_2^2 + \lambda^T \phi(x) \phi(x)^T \alpha$$

$\Downarrow$
 $K_{ij} \rightarrow$ kernel matrix
 * non linear distance b/w points
 * covariance b/w the $\phi(x)$'s all training

$K_{n \times n}$ is non linear distance b/w all training points.

$$K_{n \times n} \rightarrow K_{i,j} = k(x_i, x_{i'}) \quad \text{kernel function}$$

Kernel function is any non-linear function of norms & distances.

exm: $k(x_i, x_{i'}) = x_i^T x_{i'} \quad (\text{linear})$ This is a linear kernel which gives regular Ridge regression

This is also called Gaussian (RBF) Kernel $\rightarrow$ most common kernel

(function of $e^{\text{norm-distance}}$) $k(x_i, x_{i'}) = e^{-\|x_i - x_{i'}\|_2^2 / \sigma^2}$

Another kernel: Polynomial Kernel

$$k(x_i, x_{i'}) = (c + x_i^T x_{i'})^d$$

non linear distances

Take away

* Kernels offer a fairly easy and computationally attractive way to go from linear p dimensional space to non-linear (comp $\rightarrow n$ dim) space.

* Incredibly flexible approach; We can get any $f(x)$ we want.

Kernel Ridge Regression?

($\alpha \rightarrow n$ -dim)

$$\min_{\alpha} \|y - K\alpha\|_2^2 + \lambda \alpha^T K \alpha$$

*

$$\hat{\alpha} = (K + \lambda I)^{-1} y$$

* closed form

- * unique
- * global form
- * exists always $\lambda > 0$
- * nonlinear

CLASSIFICATION

It involves the process of categorizing or labeling data points into predefined classes or categories based on their features.

$y_i \in \{0, 1\}$ or $\{-1, 1\}$ $i = 1 \dots n$ } Binary classification: deciding bw 2 classes.

$y_i \in \{1 \dots K\}$ $i = 1 \dots n$ } Multi-class Classification:
 $K = 1 \dots K$ } K : categories or classes

Two types of Classifiers:

1) Discriminative Classifiers: Goal: to separate classes via decision boundary. Learn the class boundary between 2 classes. So, if the discriminative classifier is linear, then it means that there are linear decision boundaries. You can draw a line bw classes.

Linear $\rightarrow$ linear decision boundaries

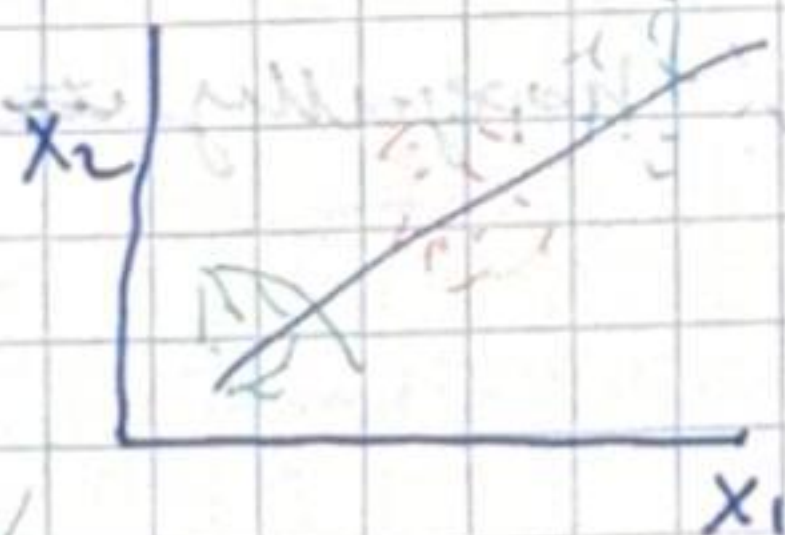
2) Generative Classifiers: These assume a probabilistic model for X given class IK .

$$X | IK$$

* Why can't we use regression? Why can't we minimize MSE loss on y ?

(i) Predicted outcomes can be outside the desired range of $[0, 1]$

(ii) If you have multiple classes, MSE loss makes no sense.



For binary Classification, MSE loss isn't terrible

$\hookrightarrow$ this is called Brier scoring

LOGISTIC REGRESSION:

Statistical and machine learning algorithm used for binary & multi class classification.

from:

Generalized Linear Model (GLM)

1) GLM(exm) $p(y|x) \sim N(\beta_0 + x^T \beta, \sigma^2 I)$

$p(y|x)$ is distributed normal

special case example

$E[y|x] = \beta_0 + x^T \beta \rightarrow$ like Linear regression with MSE loss

this leads us to Linear Regression with MSE Loss

* LINKS

* GLMs:

$p(y|x) \sim$ comes from Exponential Family Distribution

$E[y|x]:$

$E[\text{Link}(y|x)] = \beta_0 + x^T \beta$

hyperplane

$\rightarrow$ linear

assume linear

$\rightarrow$ non-linear transform that is appropriate for different Y s.

Generalized linear model, GLM, Taking a ~~linear~~ normal model, which assumes $y|x$ is linear, and apply a non-linear transform to this ~~link~~, and this link is going to be determined by what distributional form you use.

We assume $E[\text{Link}(y|x)]$ is linear, the transformation is going to help us work with binary wise / categorical wise.

Binary Classification:

$y_i = \{0, 1\} \quad i = 1, \dots, n$

Logistic: $E[\text{logit}(y|x)] = \beta_0 + x^T \beta$

$= \log \left[\frac{p(y=1|x)}{p(y=0|x)} \right] = \beta_0 + x^T \beta$

Log-odds

(log odds function)

$P(x) = P(y=1|x);$

$\log \left[\frac{P(x)}{1-P(x)} \right] = \beta_0 + x^T \beta$

* What values can the log-odds (logit) take?

$$\begin{aligned} \text{if } p(x) \rightarrow 0, \text{ logit} &\rightarrow -\infty \\ p(x) \rightarrow 1, \text{ logit} &\rightarrow +\infty \end{aligned}$$

} $\mathbb{R}$ this takes on values in the whole real line

The function takes values 0s and 1s and we get a line as an output.

* What is $p(x)$?

Solve $\log \left[\frac{p(x)}{1-p(x)} \right]$ for $p(x)$;

$$p(x) = \frac{e^{x^T \beta}}{1 + e^{x^T \beta}} = \frac{1}{1 + e^{-x^T \beta}}$$

*** sigmoidal function

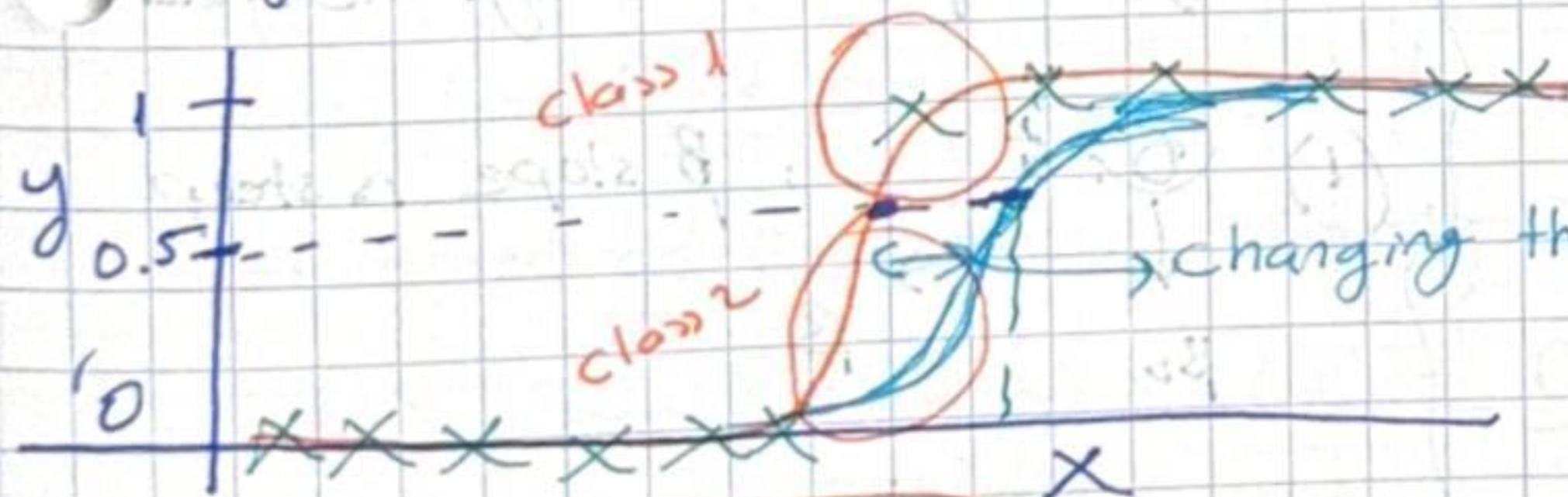
$$\sigma(x^T \beta)$$

$$\text{if } x \rightarrow \infty, p(x) \rightarrow 1$$

$$\text{if } x \rightarrow -\infty, p(x) \rightarrow 0$$

$$p(x) \in [0, 1]$$

zeros and 1s



Sigmoid function

with intercept, the function

$$p(x) = \frac{1}{1 + e^{-\beta_0 - x^T \beta}}$$

$$\beta \in \mathbb{R}^p$$

for each feature we have a β .

changes the decision boundary.

Yes

* Do we need an intercept?

Shifting the intercept, changes the range where the sigmoid

fits 0.5. Below 0.5, class 0, Above 0.5, class 1.

It is important to have an intercept, because it shifts over

the predictive value. It changes the level at which we are

to predict class 0, or class 1.

$$(*) \sigma(x^T \beta) (*)$$

$(*)$ Sigmoid function takes values from $\mathbb{R}^p$,
 which is x , And outputs to
 sigmoid function
 $x \rightarrow \infty, p(x) \rightarrow 1$
 $x \rightarrow -\infty, p(x) \rightarrow 0$

$$p(x) \in [0, 1]$$

Logit

$$\log \left[\frac{p(x)}{1-p(x)} \right] = \beta_0 + x\beta$$

Changing β will change the

$(*)$ Changing β changes the slope of the curve, which is how quickly from class 0, to 1.



$$p(x) = \frac{1}{1 + e^{-\beta_0 + x^T \beta}}$$

If we do not want to parameterize β_0 ,

- i) add a column of 1's to $X \rightarrow \tilde{X}$
- ii) Center the columns of X .

$(*)$ (we do not center y , it's zero and 1)

for this binary classification;
Prediction: When we have a new data point, x^* , we simply compute the probability.

$$p(x^*) = \frac{1}{1 + e^{-x^{*T}\hat{\beta}}}, \text{ and our } \hat{y} = \text{round}(\hat{p}(x^*))$$

(*) hard labels predictions

(*) soft labels predictions

Decision boundary:

$$p(y=1 | x) = p(y=0 | x) = 1/2$$

$$\log\left(\frac{p(x)}{1-p(x)}\right) = \log(1) = 0 = x^T \beta$$

we get this as our decision boundary. Which is a hyperplane. Which is linear.

Even if we use non linear transform, (logist), we get back a linear decision boundary between the classes.

So, this is a discriminative classifier. (Learning the boundary)

[Lab 8]

(*) Where does Logit come from and how do we know we use this?

GLM based on Exponential Family Distribution:

$$p(z) = \exp \left\{ \underbrace{\theta \beta(z)}_{\text{sufficient statistic}} + \underbrace{C(z)}_{\text{base}} + \underbrace{D(\theta)}_{\text{log partition}} \right\}$$

(0, 1 random variables)
 $\downarrow$
 Bernoulli

Examples:

{ Normal, Gamma, Exp, Bernoulli, Binomial, Chi-Squ, Poisson... }

$P(y|x) \sim \text{Bernoulli}(p(x))$ assume this distributional form

$$= \exp \left[\log \left[p(x)^y (1-p(x))^{(1-y)} \right] \right]$$

$$= \exp \left[y \log p(x) + (1-y) \log (1-p(x)) \right]$$

$$= \exp \left[\underbrace{y \log \left(\frac{p(x)}{1-p(x)} \right)}_{\text{log odds}} + \underbrace{\log (1-p(x))}_{p(\theta)} \right]$$

$\beta(z)$ θ

* Estimation for Logistic Regression Models fitting the model *

For probabilistic ^{models} methods, our estimator will be MLE. ^{maximum likelihood estimator}

$l(\beta) \rightarrow$ log Likelihood

$\rightarrow$ likelihood function

$* l(\beta) = \sum_{i=1}^n y_i (x_i^T \beta) - \log(1 + e^{x_i^T \beta}) *$

Maximize the likelihood function, its equivalent to minimizing the negative log likelihood.

$$\max_{\beta} l(\beta) \Leftrightarrow \min_{\beta} -l(\beta) \rightarrow \text{convex!}$$

Differentiate

$$\frac{\partial l}{\partial \beta} = \sum \left[y_i x_i - x_i^T \frac{e^{x_i^T \beta}}{1 + e^{x_i^T \beta}} \right] \rightarrow p(x_i)$$
$$= \sum x_i^T [y - p(x_i)]$$

No closed form solution, So we can do Gradient descent.

Gradient Descent; When we have ~~good~~ second derivatives which have nice form, and we have Gaussians which has nice form. we can use Newton's Method, (2nd order)

Newton's Method: (2nd order method)

* They converge very quickly *

* For Logistic regression, fitting $l(\beta)$ is typically done by Newton's method, bc it is a nice second derivative. It can be ~~overweighted~~ written as Iteratively Re-weighted Squares.

Logistic regression

$\rightarrow$ Fast(ish) + Easy

$\rightarrow$ Newton's Method

Iterative Re-weighted Least Squares

* fast ish + easy

Can we do Multi-Class Logistic Regression?

$$y_i = \{1, \dots, K\}, \quad i = 1, \dots, n$$

Multinomial Distribution:

fit MLE

Derive the exponential form and GLMs;

$K-1$ Formulation (Multinomial):

$$\log \left[\frac{p(y=k|x)}{p(y=k|x)} \right] = x^T \underbrace{\beta^{(k)}}_{\beta^{(k)} \text{ } p \times k-1}, \quad k=1 \dots K-1$$

$$p(y=k|x) = \frac{e^{x^T \beta^{(k)}}}{1 + \sum_{l=1}^K e^{x^T \beta^{(l)}}}$$

How do we interpret this model?

What's log odds for apples vs peer, oranges vs pears...

$$p(y=k|x) = \frac{1}{1 + \sum_{l=1}^K e^{x^T \beta^{(l)}}}$$

[lab 8] We lose the interpretation of parameters.

Another way of doing this:

K Formulation: (Soft max formulation) fit MLE
we will define this by the probability of being in class K ;

$$p(y=k|x) = \frac{e^{x^T \beta^{(k)}}}{\sum_{l=1}^K e^{x^T \beta^{(l)}}}$$

softmax function

Probability of K normalized by all other probabilities of the other classes.

Soft max, gives the interpretable probabilities for each class that sum to 1.

If we have 4 classes; $\begin{bmatrix} 0.4 & 0.25 & 0.05 & 0.3 \end{bmatrix}$

1. (*) Probabilities are interpretable

2. Parameters are interpretable; organize parameters:

$$B_{p \times K} \rightarrow$$

B_{jk} is the slope associated with feature j in the k^{th} class.

So, this is the slope of that logistic function for predicting feature j as a member of k^{th} class. If that belongs to there.

If we have new data x^* , for prediction we would calculate P_k :

$$P_k(x^*)$$

soft max function

hard function.

Then our hard label is:

$$\hat{y} = \arg \max_k \hat{P}_k(x^*)$$

* we lose the log-odds GLM interpretation.

[Lab 8]

[17]

glmnet

(directly interpretable, Alcohol, class 1, class 2)

Review

(1) Logistic Regression: ~~the~~ classification method, that we do binary classification.

$$y_i \in \{0, 1\} \quad i = 1, \dots, n$$

$$E[\text{link}(y|x)] = \beta_0 + x^T \beta \quad \leftarrow \text{GLMs}$$

for logistic regression link function is logit or log-odds

$$P(y=1|x) = P(x) = \frac{1}{1 + e^{-\beta_0 - x^T \beta}} \quad \left\} \text{sigmoid function.}$$

Sigmoid function takes real values in $\mathbb{R}^T$ and outputs in the range 0-1, allowing us to fit hyperplanes on real line.

* What assumption do we have about our data to get this link function?

→ We assume Bernoulli distribution, and that is how we derive logit or log-odds function.

* How do we fit this logistic regression function? What is our estimation strategy here?
MLE, particularly

* Newton's method

(2) Multinomial Regression

$$y_i \in \{1, \dots, K\} \quad \text{we have } K \text{ different categories (labels)}$$

$K-1$ Formulation, GLM assumes multinomial distribution.

How would you interpret this: It was the log-odds of the k -class compared to

The link function comes to this:

$$E \left[\log \left[\frac{p(y=k|x)}{p(y=k|x)} \right] \right] = \beta_0 + x^T \beta^{(k)} \quad k=1 \dots K-1$$

* What is the issue with this formulation?

i) Lacks interpretability.

- Probabilities are hard to interpret. 1v3 2v3
1v2 etc.
- Parameters themselves are challenging to interpret.

③ Soft-Max or K-formulation

We directly define the probability $P(y=k|x)$

$$P(y=k|x) = \frac{e^{x^T \beta^{(k)}}}{\sum_{l=1}^K e^{x^T \beta^{(l)}}} \quad \left. \begin{array}{l} \text{probabilities of} \\ \text{the other classes.} \end{array} \right\} \text{soft max function.}$$

We call this soft-max function because it outputs a vector of probabilities. Like this exm:

$$\begin{bmatrix} 0.55 & 0.05 & 0.05 & 0.35 \end{bmatrix} \quad \text{output probabilities not only hard labels.}$$

Prediction, $\hat{p}_k(x^*)$, soft prediction / label

$\hat{y} = \underset{k}{\operatorname{argmax}} p_k(x^*)$, hard label

*** The probabilities are interpretable

*** Our parameters are $B_{p \times K}$ matrix ($p \times K$)

$B_{p \times K} \rightarrow B_{jk}$ is the slope of the j^{th} feature for the k^{th} class

You can see which features are predictive for which class.

* You cannot do this with multinomial formulation

④ Regularized Logistic & Softmax

$\{ \text{if } p \gg n \text{ and } K \text{ is } \uparrow\uparrow \text{ huge} \} \Rightarrow \underline{\text{we do regularization}}$

$\min_{\beta} -\ell(\beta) + \lambda P(\beta) \rightarrow$ add penalty term when $p \uparrow, k \uparrow$

When $p \uparrow$ and when multi class, $\# \uparrow$

What kind of penalties do we add?

i) L_1 or lasso penalty if we want to use feature selection.

④ iv) l_2 or ridge penalty if we want to do prediction, while especially P and K are large.

GROUP LASSO:

Softmax: $B_{P \times K}$ Consider the softmax case where we have $B_{P \times K}$

When goal is aid in interpretability and we want to do feature selection.

[Lab] [21] L1 softmax

- L_1 doesn't select features for all classes
- You can do group Lasso (textbook) This select the some features for all classes

[Lab] [20]

⑤ Bayes Classifier & LDA:

General idea for Bayes Classifiers: (?)

Bayes classifiers are generative classifiers.

Generative Classifier: Bayes Classifier: $P(u)$

Bayes' theorem:

$$P(Y=k|X) = \frac{P(Y=k) P(X|Y=k)}{P(X)}$$

Assume: $P(X | Y = k)$
we assume this and want

Want: $p(y=k \mid x)$
want to predict y given the data

→ Bayes Rule

⑥ Naive Bayes:

We assume that probability of x given y equals to k ;

$$P(x|y=k) \sim N(\mu_k, \sigma^2 I)$$

Gaussian
distribution

centroid
for class k

identity: $\sigma^2 I$
spherical type
covariance

We just plug in the Gaussian distribution directly in Bayes rule.

We plug in those probabilities, we get our probabilities, simplify.

* And How do we actually fit the naive bayes classifier?

We do MLE; we fit using MLE, estimating the means, priors and σ^2 variances.

We plug the MLE estimates for $\hat{\mu}$ and $\hat{\sigma}$ into our Bayes rule classifiers. And that will give us these estimated probabilities.

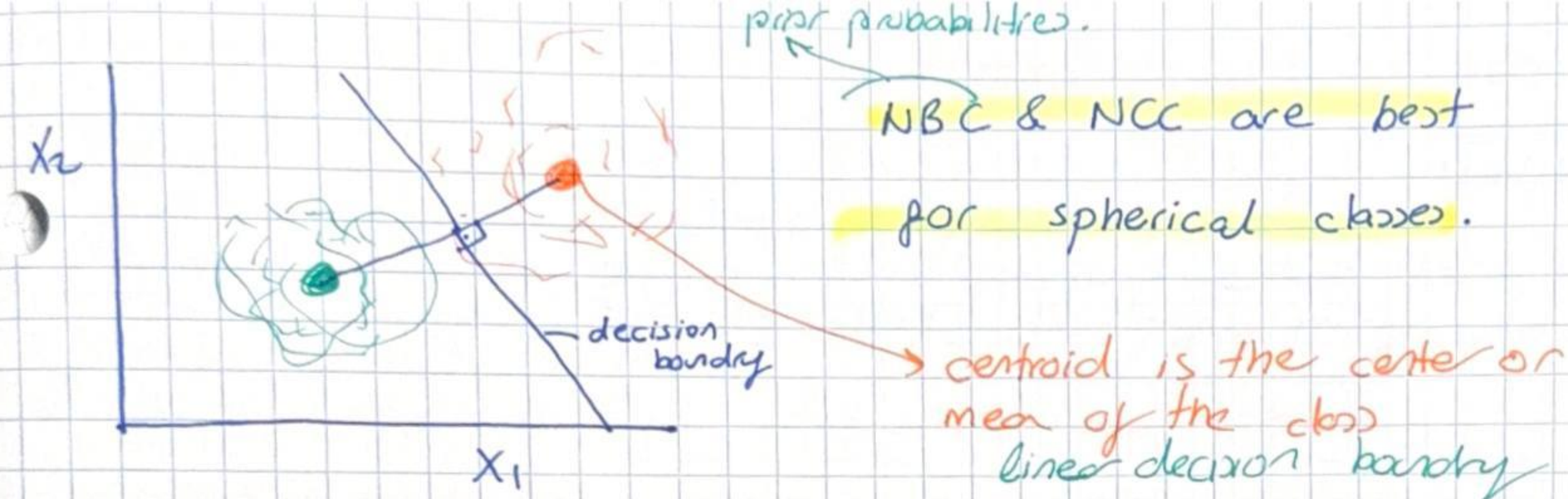
Just like soft max, now we have a separate probability for each class k and we can have a soft label which is the probability or we can classify to the class with the highest probability and have a hard label.

* Under certain assumptions;

(i) Euclidean distances and

(ii) NBC has equal π 's in each class.

NBC $\Leftrightarrow$ NCC
nearest
centroid
classifier



Decision rule is to classify to the class nearest to each point.

⑦ Linear Discriminant Analysis.

When you have correlation; Naive bayes classifier is not the best.

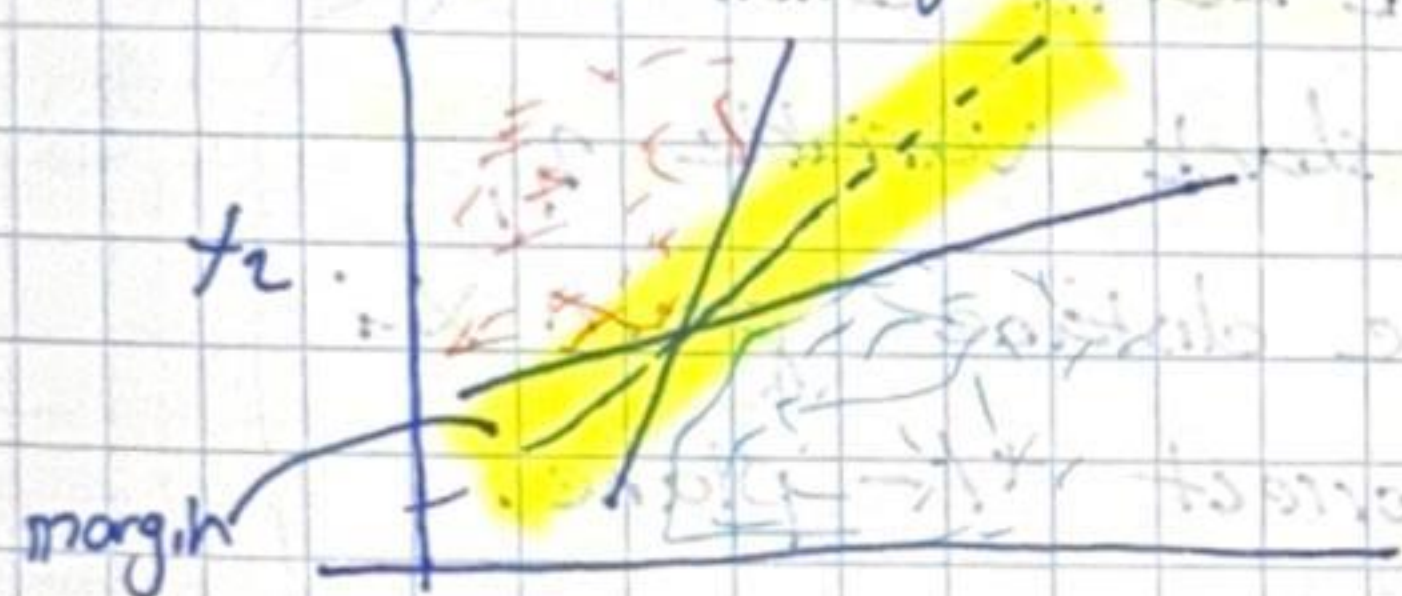
$$p(x | y=k) \sim N(\mu_k, \Sigma_w) \rightarrow \text{allow correlation between features.}$$

Geometric - Inspired Classifiers: Discriminative Classifiers, which measure the margin between classes. Sometimes, when we learn the decision boundary, these are called the large margin, or max margin classifier.

↳ Large margin classifier

BINARY CLASSIFICATION

Binary classification: $y_i \in \{-1, 1\}$ $i=1, \dots, n$



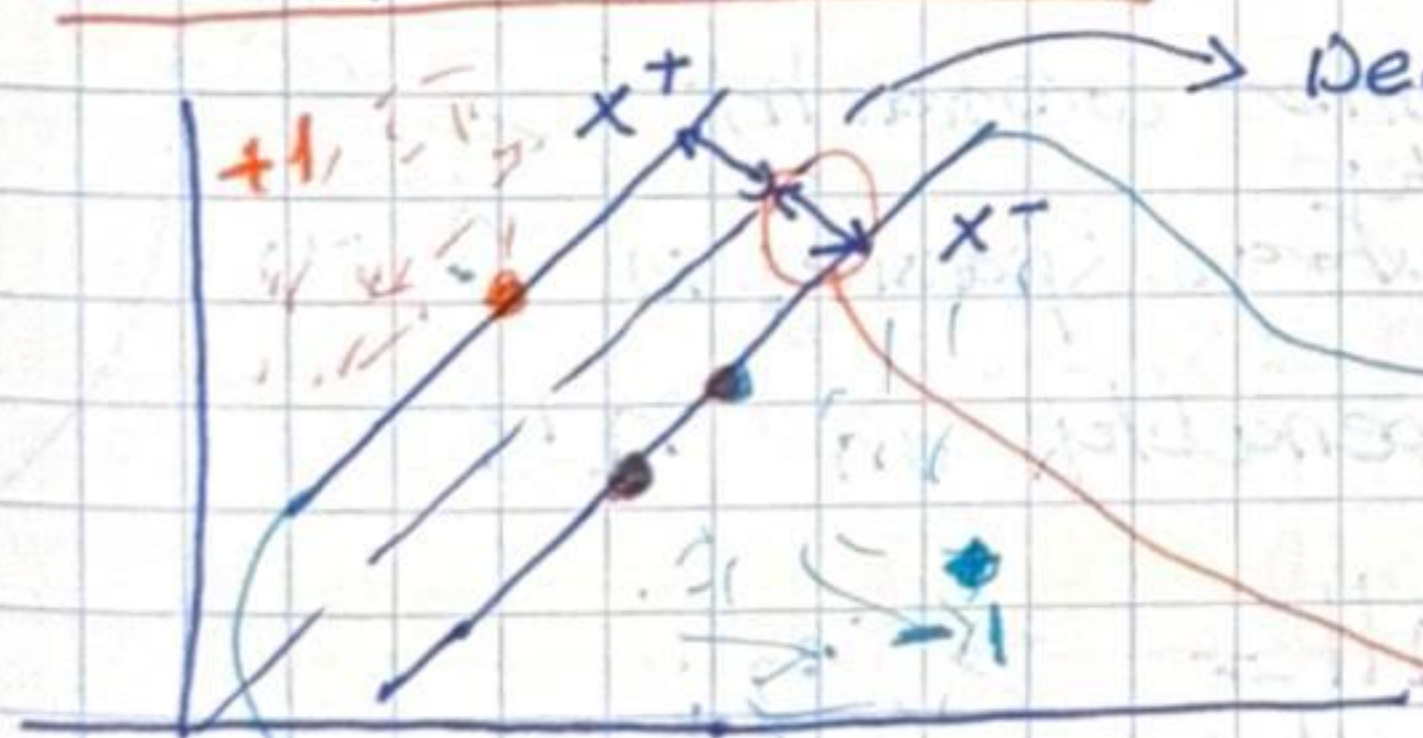
Case: The two classes are linearly separable.

So with high probability, there are infinitely many lines, hyperplanes that separate the two classes.

Optimal
Separating
Hyperplanes (OSH)

Idea: Make the margin b/w the two classes as large as possible, that means Fit the largest possible margin. Why? So our decision boundary is at the center of the margin.

Geometric Derivation:



Decision Boundary

$$f(x) = \beta_0 + x^T \beta = 0$$

"minus plane"

$$f^-(x) = -1 = \beta_0 + x^T \beta$$

$$f^+(x) = +1 = \beta_0 + x^T \beta$$

M = margin size

$$\text{dot}(x^-, x^+) = 2M$$

* Slope of the line connecting nearest x^+ & $x^- \rightarrow -1/\beta$
and direction $\Rightarrow \frac{\|\beta\|_2}{\beta}$

solve for M:

$$x^+ = x^- + 2M \frac{\|\beta\|_2}{\beta} \Rightarrow M = \frac{1}{\|\beta\|_2}$$

Optimization Problem:

(maximize the distance)

max m subject to data outside ± 1 planes.

$$\max_{\beta} m = \max_{\beta} \frac{1}{\|\beta\|_2}$$

$$\text{subject to } \left. \begin{array}{l} \beta_0 + x_i^T \beta \geq 1 \quad \text{if } y_i = 1 \\ \beta_0 + x_i^T \beta \leq -1 \quad \text{if } y_i = -1 \end{array} \right\}$$

Optimization Problem for
Optimal separation Hyperplane

$$\Rightarrow \min_{\beta} \|\beta\|_2^2$$

$$\text{subject to } y_i(\beta_0 + x_i^T \beta) \geq 1, i = 1, \dots, n$$

Solving the Optimization Problem: It is strictly convex, which means it has unique global solution.
- It has no closed form solution; iterative up.

Interpretation: How many data points are on the margins?
like 2-3 here;

These are called support vectors ∇ (they hold up the margins)

$$\{x_i : |x_i^T \beta + \beta_0| = 1\}$$

* When do OSH fail?

- They are not very robust, they are very dependent on the training data, which means they don't generalize well (when not linearly separable)

* How can we fix OSH? When they are not linearly separable?

SVM

we want η_i to be sparse, we
like most of them be zero.

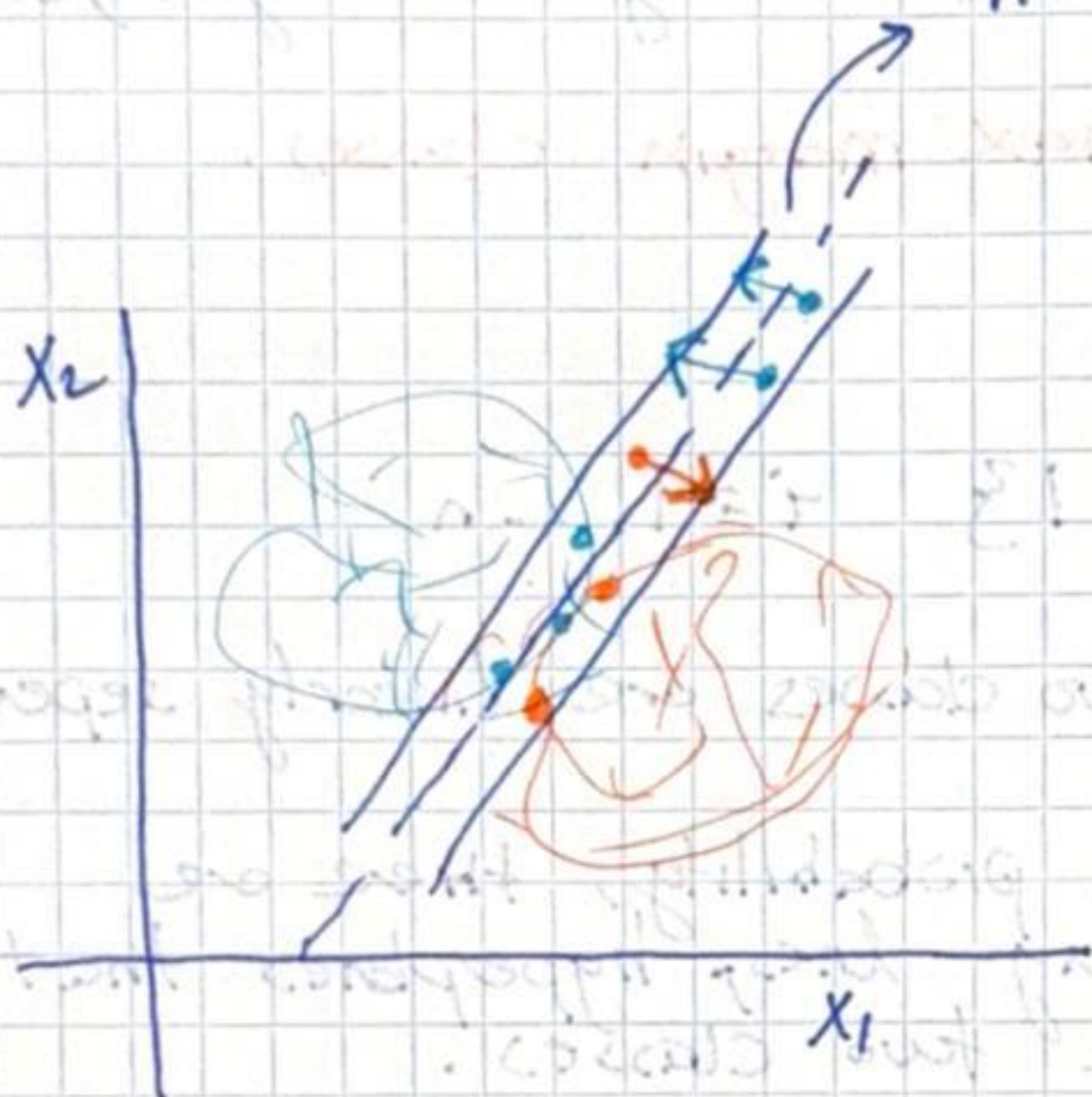
non-zero η_i 's

How can we expect the data,
if there are many points in the
margin?

idea: Relax constraint

via a slack variable η_i

η_i is the distance from x_i
to the correct $+/-$ plane.



$$\min_{\beta_0, \beta, \eta} \|\beta\|_2^2 + C \sum_{i=1}^n \eta_i$$

cost parameter, to encourage sparsity.

subject to $y_i(\beta_0 + \beta^T x_i) \geq 1 - \eta_i, \eta_i \geq 0$

Need another constraint
or penalty
to encourage sparsity in η_i 's
→ Lasso penalty on η 's

$$\|\eta\|_1 = \sum \eta_i$$

$\beta \in \mathbb{R}^p \rightarrow$ slope of the decision boundary

$\eta \in \mathbb{R}^{nt} : \text{slack variable}$

Hyperparameters: C

$C \uparrow ; \# \text{ non-zero } \eta_i \uparrow, \text{ margin } \downarrow$

$C \downarrow ;$

$\uparrow,$

* C parameter controls the margin size. Tune via C.V.

Interpret Training Data

if $\hat{\eta}_i = 0 \Rightarrow x_i$ is well classified (correct side +/- plane)

Support vectors: **SV** or **influence** or **leverage points**

SVs $\left\{ \begin{array}{l} \{x_i: |x_i^T \hat{\beta} + \hat{\beta}_0| = 1\} \Rightarrow \text{on } \pm \text{ plane} \\ \{x_i: 0 < \hat{\eta}_i \leq 1\} \Rightarrow \text{correctly classified but in margin} \\ \{x_i: \hat{\eta}_i > 1\} \Rightarrow \text{misclassified \& outside the margin.} \end{array} \right.$

Solving Linear SVMs: Similar to optimal separation Hyperplanes, SVMs are **strictly convex**, which means we have **global unique solutions**.

Prediction: $x^* \rightarrow \hat{f}(x^*) = \hat{\beta}_0 + x^{*T} \hat{\beta}$

$\hat{y} = \text{sign}(\hat{f}(x^*)) \rightarrow \text{hard label}$ (sign of $\hat{f}$ gives us the hard labels)

if $|\hat{f}(x^*)| \uparrow$, it is far away from the margin, which means it is more confident in prediction.

Summary of SVMs:

- * It tends to be more robust than logistic, NB, or LDA.
- * It handles departures from a given distribution, comes from geometry.

Multi Class Classification:

$y_i \in \{1, \dots, K\}$, How?

(i) **One vs One**: Class k vs class k'
 $\binom{K}{2} \rightarrow$ computationally burdensome

Prediction is done by taking the majority vote.

(ii) **One vs All (Rest)**: Class k vs class not k

K total classifiers

prediction?

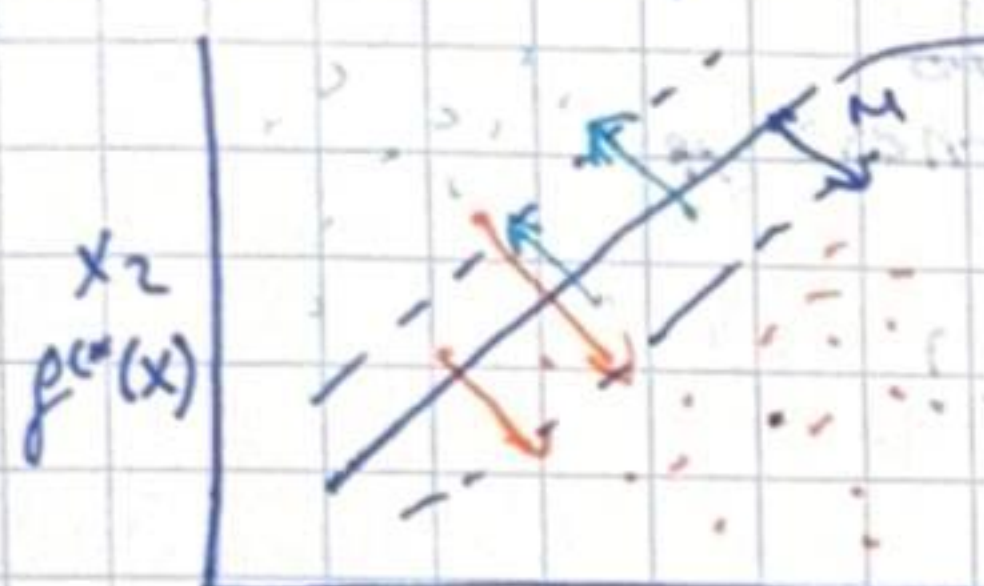
$\hat{f}_k(x^*)$; $k=1, \dots, K$

It is the class farthest from the margin

$\hat{y} = \arg \max_k \hat{f}_k(x^*)$

Binary Classification: $y_i \in \{-1, 1\}$

Max-Margin Classifier: Discriminative classifier where we directly seek to learn the decision b.d.



$f(x) = \beta_0 + x^T \beta = 0$ between two classes.

$\mu = 1 / \|\beta\|_2$

Optimal separating Hyperplanes: DSH

Linearly separable

$\min_{\beta_0, \beta} \|\beta\|_2^2$

subject to the data living outside the margin.

subject to $y_i(\beta_0 + x_i^T \beta) \geq 1$

For non-separable: We use slack variable

$\min_{\beta_0, \beta, \eta} \|\beta\|_2^2$

$+ C \sum \eta_i$

sub to $y_i(\beta_0 + x_i^T \beta) \geq 1 - \eta_i$

Hyperparameter: $C \uparrow$ margin $\downarrow$
 $C \downarrow$ margin $\uparrow$

* Interpretation of features $\Rightarrow \hat{\beta}_j$, slope of $\hat{\beta}_j$

Interpretation of observation $\Rightarrow$ Support Vectors!
Set of observation that are on or inside of the margin.

* SVMs as Empirical Risk Minimization

$\min_{\beta_0, \beta} \sum_{i=1}^n [1 - y_i(\beta_0 + x_i^T \beta)]_+ + \lambda \|\beta\|_2^2$

Ridge

for some $C \Leftrightarrow \lambda$, $1 \Leftrightarrow 2$

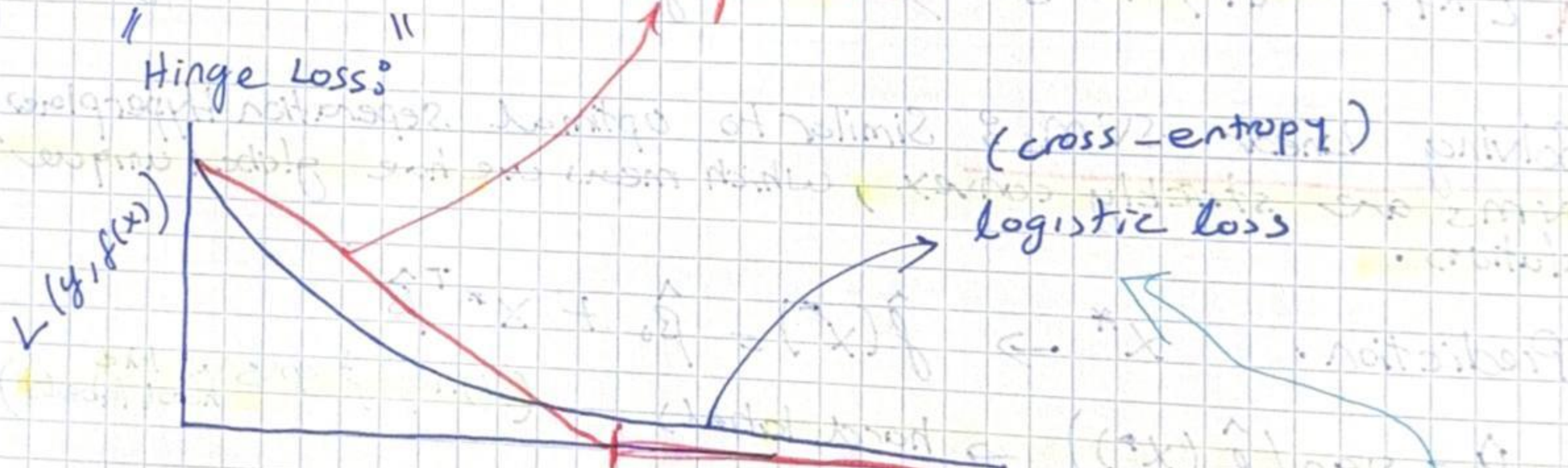
if x is well classified (correct side of margin) $\Rightarrow$ $y = 0$

if x is not well classified (wrong side of margin) $\Rightarrow$ $y = 1$

if x is on the margin $\Rightarrow$ $y = 0.5$

$$\sum_{i=1}^n [1 - y_i (\beta_0 + x_i^T \beta)] \geq \gamma$$

if $\gamma > 0$ then the margin is at least γ



margin $\uparrow$ margin $\downarrow$

if the margin is large, the loss is small. If the margin is small, the loss is large.

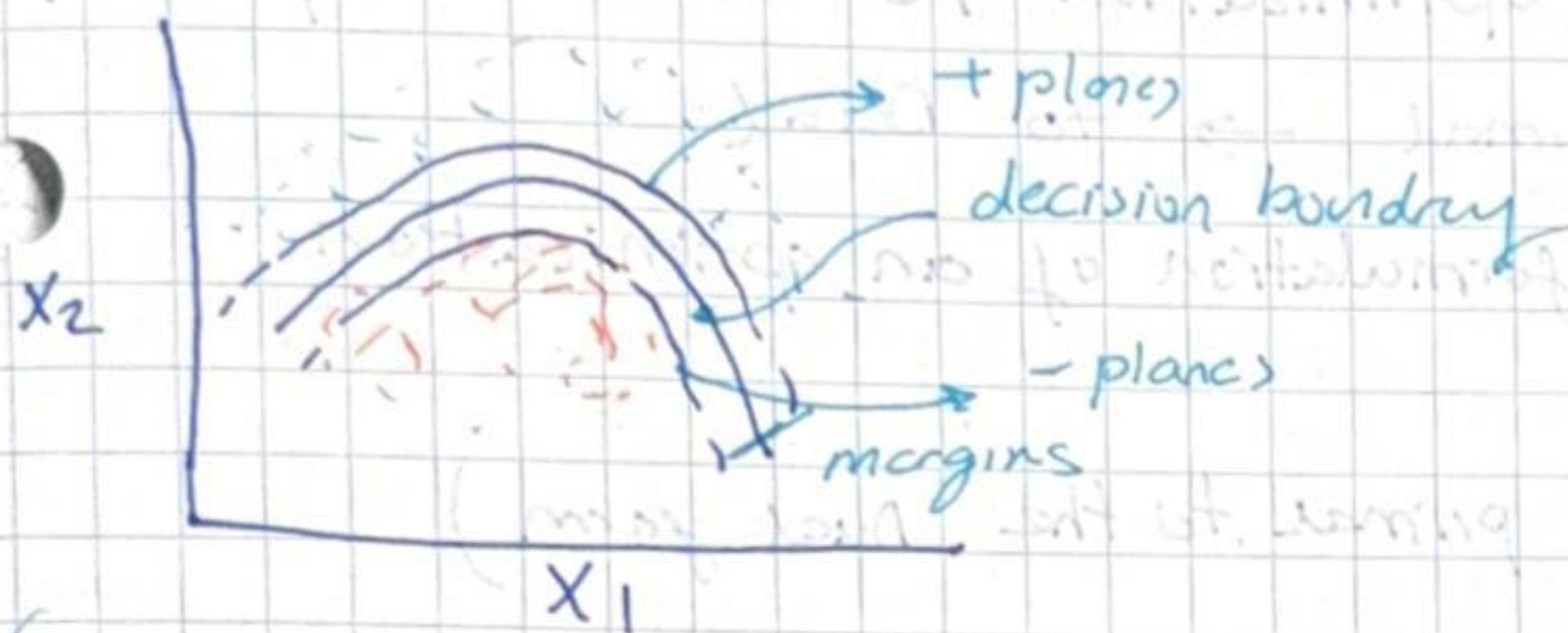
The Hinge Loss is used in Support Vector Machines. The Logistic Loss is used in Logistic Regression.

The Hinge Loss is not differentiable at the margin. The Logistic Loss is differentiable everywhere.

The Hinge Loss is a convex function. The Logistic Loss is also a convex function.

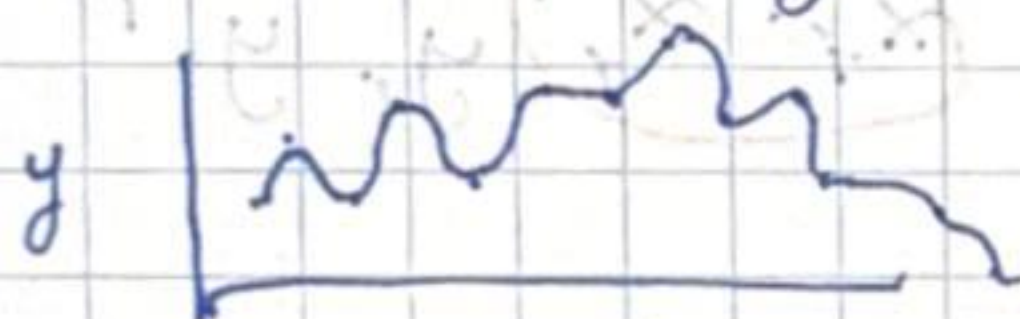
Kernel SVMs

Non linear Decision boundaries



Kernels Review: $f(x) = \sum_{i=1}^{\infty} c_i \phi_i(x)$ Infinite dim basis function

Mercer's Representer Theorem: To fit infinite dimensional functions we need to only evaluate them on the N training points.



Need to go from p to n dim space

Zero training Error

Kernel Trick: We only need to worry about the covariance b/w $f(x)$ and $f(z)$:

$$* \text{cov}(f(x), f(z)) \triangleq K(x, z),$$

* Kernels are like "lifted" inner products, or non linear versions of inner products.

exms

$$K(x, z) = x^T z : \text{linear}$$

$$K(x, z) = (c + x^T z)^d : \text{polynomial}$$

$$K(x, z) = e^{-\|x - z\|_2^2 / \gamma} \rightarrow \text{RBF}$$

Kernels are typically non-linear functions of inner products, norms or distances.

* To apply the kernel trick, from linear to non linear in a very computationally efficient manner; Two tricks:

1) Look at the change in variable, from p -dim to n dim.

$$\|y - X\beta\|_2^2, \quad \alpha = X^T \beta \quad n \times 1 \text{ vector}$$

$$\|y - K \alpha\|_2^2 + \lambda \alpha^T K \alpha$$

$K_{n \times n}$ Kernel matrix $w / K_{ii} = K(x_i, x_i)$

10/3/23 Review Kernel Trick

Kernel is the computationally efficient way of fitting infinite dimensional functions - non-linear function.

* Mercer's Rep. Theorem

* $\infty \rightarrow n$

Efficient way from non-linear to linear.

* How do we use the Kernel trick?

When we write our linear regression models we write them in p -dim space, with p -dimensions.

Goal: Going from p dim to n dim, and replace the inner products $x^T z$ with kernels

1- change variables

$$\beta = X^T \alpha \quad (\alpha \in \mathbb{R}^n)$$

2 - Dual problem

replace $x^T z$ with (XZ^T)

Common Kernels

RBF, Polynomial

#

mm